

**Федеральное государственное образовательное бюджетное
учреждение высшего образования
«Финансовый университет при Правительстве Российской Федерации»
(Финансовый университет)**

Колледж информатики и программирования

ОТЧЁТ
По учебной практике

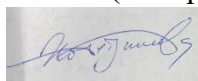
Специальность 09.02.07 «Информационные системы и программирование»
(код) (наименование)

Профессиональный модуль ПМ.01 Разработка модулей программного обеспечения для компьютерных систем
(код) (наименование)

Междисциплинарный курс МДК.01.03 Разработка мобильных приложений
(код) (наименование)

Выполнил:

Студент 4 курса 4ИСИП-522 учебной группы
(номер) (номер)



(подпись)

А. И. Хайретдинова
(инициалы, фамилия)

Проверил:

Руководитель практики от Колледжа
информатики и программирования

Преподаватель
(квалификационная
категория или звание,
должность)

Д.А. Поволяев
(инициалы, фамилия)

Москва – 2026

Перечень работ, выполненных в ходе учебной практики

№ п/п	Виды работ	Оценка
1	Планирование проекта для разрабатываемого мобильного приложения	Отлично
2	Разработка спецификаций мобильного приложения	Отлично
3	Проектирование мобильного приложения	Отлично
4	Разработка мобильного приложения	Отлично
5	Отладка и тестирование мобильного приложения	Отлично
6	Разработка технической документации мобильного приложения	Отлично

ВВЕДЕНИЕ

В рамках учебной практики была разработана полнофункциональная система для управляющей компании на платформе WPF с использованием Microsoft SQL Server. Приложение предназначено для автоматизации процессов обработки заявок на ремонт и обслуживание от жильцов, обеспечивая полный цикл управления.

Система реализует базовый функционал подсистемы работы с заявками, включая просмотр списка адресов и сотрудников, добавление, редактирование и удаление заявок с привязкой к объектам недвижимости и ответственным исполнителям. Также предусмотрена возможность анализа истории выполнения заявок по сотрудникам и адресам.

Пользовательский интерфейс выполнен в едином согласованном стиле в соответствии с руководством по оформлению, включает фирменную символику, интуитивную навигацию с возможностью возврата между формами, а также валидацию ввода и обработку исключительных ситуаций с информативными сообщениями для пользователя. Также выполнены задачи по резервному копированию базы данных, построению диаграммы прецедентов и разработке тестовых сценариев.

Итоговое решение представляет собой законченное программное обеспечение, соответствующее техническому заданию, с полным комплектом документации, включая ER-диаграмму, блок-схемы алгоритмов и тестовую документацию.

РАЗРАБОТКА БАЗЫ ДАННЫХ

Все данные приложения хранятся в базе ManagementCompanyDB, созданной с помощью Microsoft SQL Server Management Studio. База включает пять таблиц, связанные внешними ключами. Структура таблиц отражена в таблице 1.

Таблица 1 – Словарь данных базы данных

Таблица	Поле	Тип данных	NULL	Ключ	Описание
Buildings	ID	INT	Нет	PK, Identity	ID дома
Buildings	Address	NVARCHAR(300)	Нет		Адрес
Apartments	ID	INT	Нет	PK, Identity	ID квартиры
Apartments	BuildingID	INT	Нет	FK → Buildings.ID	Ссылка на дом
Apartments	ApartmentNumber	INT	Нет		Номер квартиры
Apartments	OwnerFullName	NVARCHAR(150)	Нет		Владелец
Employees	ID	INT	Нет	PK, Identity	ID сотрудника
Employees	LastName	NVARCHAR(50)	Нет		Фамилия
Employees	FirstName	NVARCHAR(50)	Нет		Имя
Employees	Position	NVARCHAR(50)	Нет		Должность
RepairRequests	ID	INT	Нет	PK, Identity	ID заявки
RepairRequests	BuildingID	INT	Нет	FK → Buildings.ID	Ссылка на дом
RepairRequests	ApartmentID	INT	Да	FK → Apartments.ID	Ссылка на квартиру
RepairRequests	ApplicantFullName	NVARCHAR(150)	Нет		Заявитель
RepairRequests	ProblemDescription	NVARCHAR(1000)	Нет		Описание проблемы
RepairRequests	Status	NVARCHAR(20)	Да		Статус заявки
RepairRequests	AssignedToEmployeeID	INT	Да	FK → Employees.ID	Ответственный
RequestHistory	ID	INT	Нет	PK, Identity	ID записи истории
RequestHistory	RequestID	INT	Нет	FK → RepairRequests.ID	Ссылка на заявку
RequestHistory	EmployeeID	INT	Нет	FK → Employees.ID	Сотрудник

Связность таблиц обеспечена внешними ключами, гарантирующими целостность данных. На рисунке 1 изображена ER-диаграмма, созданная в SQL Server Management Studio, демонстрирующая отношения один-ко-многим между таблицами Buildings и Apartments, Buildings и RepairRequests, Apartments и RepairRequests, Employees и RepairRequests, а также RepairRequests и RequestHistory.

На рисунке 1 изображена ER-диаграмма.

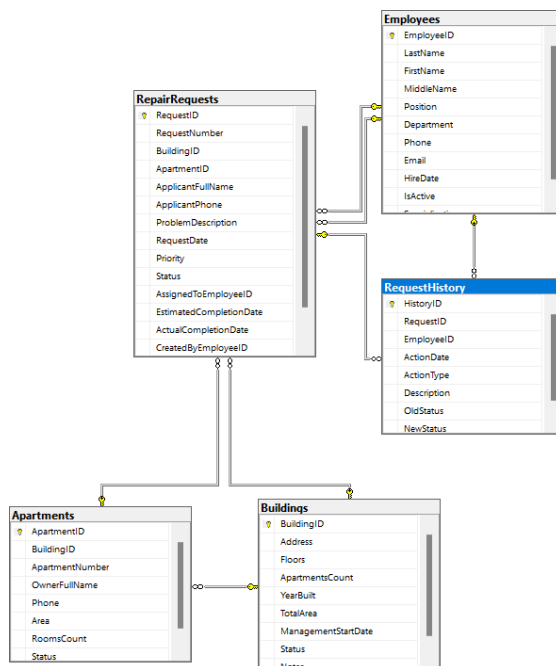


Рисунок 1 – ER-диаграмма

На рисунках 2-3 представлено содержимое таблиц.

BuildingID	Address	Floors	ApartmentsCount	YearBuilt	TotalArea	ManagementStartDate	Status	Notes
1	ул. 45 Паратень, 42, Ставрополь	9	52	2001	1978.70	2015-04-22	Активен	NULL
2	ул. Васильева, 1, Ставрополь	9	144	1983	7950.40	2015-04-22	Активен	NULL
3	ул. Докторовцев, 66/2, Ставрополь	9	102	1984	3176.40	2020-11-01	Активен	NULL
4	ул. Мира, 238, Ставрополь	10	62	1991	4423.10	2020-11-04	Активен	NULL
5	ул. Мира, 272, Ставрополь	9	88	2006	6204.70	2015-04-22	Активен	NULL
6	ул. Мира, 278, Ставрополь	10	40	2008	3294.10	2019-08-01	Активен	NULL
7	пл. Восточная, 40, Светлоград	5	70	1985	2369.20	2018-12-01	Активен	NULL
8	пл. Восточная, 43, Светлоград	5	68	1987	3702.80	2019-07-01	Активен	NULL

ApartmentID	BuildingID	ApartmentNumber	OwnerFullName	Phone	Area	RoomsCount	Status
1	1	1	Беленко Ольга Викторовна	+79180000001	49.50	2	Заселена
2	1	18	Мазалова Ирина Львовна	+79185647218	48.20	2	Заселена
3	1	18	Семеница Юрий Геннадьевич	+79180000003	42.80	1	Заселена
4	1	18	Савельев Олег Иванович	+79287815445	51.30	3	Заселена
5	1	18	Галицкий Игорь Леонидович	+79180000005	47.60	2	Заселена
6	1	18	Бунин Захар Михайлович	+79180000006	43.90	2	Заселена
7	1	18	Байцеев Павел Иванович	+79180000007	39.80	1	Заселена
8	1	18	Байцеева Агата Рустамовна	+89643324574	52.10	3	Заселена

EmployeeID	LastName	FirstName	MiddleName	Position	Department	Phone	Email	HireDate	IsActive	Specialization
1	Петров	Алексей	Иванович	Слесарь	Ремонтная служба	+79161234501	rethov@company.ru	2026-01-22	1	NULL
2	Иванова	Мария	Сергеевна	Электрик	Ремонтная служба	+79161234502	mariaiv@company.ru	2026-01-22	1	NULL
3	Сидоров	Дмитрий	Александрович	Сантехник	Ремонтная служба	+79161234503	sidorov@company.ru	2026-01-22	1	NULL
4	Козлова	Ольга	Викторовна	Диспетчер	Приемная	+79161234504	kozlova@company.ru	2026-01-22	1	NULL
5	Васильев	Сергей	Петрович	Управляющий	Администрация	+79161234505	vasiliev@company.ru	2026-01-22	1	NULL
6	Николаев	Андрей	Михайлович	Мастер	Ремонтная служба	+79161234506	nikolaev@company.ru	2026-01-22	1	NULL
7	Федоров	Елена	Александровна	Бухгалтер	Финансовый отд.	+79161234507	fedorova@company.ru	2026-01-22	1	NULL

RequestID	RequestNumber	BuildingID	ApartmentID	ApplicantFullName	ApplicantPhone	ProblemDescription	RequestDate	Priority	Status	AssignedToEmployeeID	EstimatedCompletionDate	ActualCompletionDate
1	Z-20260122-6882	18	1	Беленко Ольга Викторовна	+79180000001	Протекает кран на кухне	2026-01-21 13:58:27.073	1	Заявка закрыта	2	2026-01-22	2026-01-22
2	Z-20260122-5955	18	2	Мазалова Ирина Львовна	+79185647218	Не работает розетка в в.	2026-01-20 13:58:27.077	2	Заявка закрыта	3	2026-01-21	2026-01-21
3	Z-20260122-5158	18	3	Семеница Юрий Геннадьевич	+79180000003	Забит слот в распределит.	2026-01-19 13:58:27.077	3	Заявка закрыта	1	2026-01-20	2026-01-20
4	Z-20260122-4205	18	4	Савельев Олег Иванович	+79287815445	Треснула в стене на балк.	2026-01-18 13:58:27.077	1	В работе	2	2026-01-25	NULL
5	Z-20260122-8702	18	5	Галицкий Игорь Леонидович	+79180000005	Не закрывается входн.	2026-01-17 13:58:27.080	2	В работе	3	2026-01-25	NULL
6	Z-20260122-7113	18	6	Бунин Захар Михайлович	+79180000006	Протекает радиатор о.	2026-01-16 13:58:27.080	3	В работе	1	2026-01-25	NULL
7	Z-20260122-7623	18	7	Байцеев Павел Иванович	+79180000007	Сломан выключатель в в.	2026-01-15 13:58:27.080	3	Открыта	2	NULL	NULL
8	Z-20260122-7486	18	8	Байцеева Агата Рустамовна	+89643324574	Запах из канализации	2026-01-14 13:58:27.080	3	Открыта	3	NULL	NULL

Рисунок 2 – Содержимое таблиц

	HistoryID	RequestID	EmployeeID	ActionDate	ActionType	Description	OldStatus	NewStatus
1	1	1	2	2026-01-21 13:58:27.073	Создание заявки	Заявка создана диспетчером	NULL	Заявка закрыта
2	2	2	3	2026-01-20 13:58:27.077	Создание заявки	Заявка создана диспетчером	NULL	Заявка закрыта
3	3	3	1	2026-01-19 13:58:27.077	Создание заявки	Заявка создана диспетчером	NULL	Заявка закрыта
4	4	4	2	2026-01-18 13:58:27.077	Создание заявки	Заявка создана диспетчером	NULL	В работе
5	5	5	3	2026-01-17 13:58:27.080	Создание заявки	Заявка создана диспетчером	NULL	В работе
6	6	6	1	2026-01-16 13:58:27.080	Создание заявки	Заявка создана диспетчером	NULL	В работе
7	7	7	2	2026-01-15 13:58:27.080	Создание заявки	Заявка создана диспетчером	NULL	Открыта
8	8	8	3	2026-01-14 13:58:27.080	Создание заявки	Заявка создана диспетчером	NULL	Открыта

Рисунок 3 – Содержимое таблиц

Полный скрипт создания базы данных с таблицами и заполнением тестовыми данными представлен в листинге 1.

Листинг 1 – SQL-скрипт создания и заполнения базы данных

```

USE master;
GO
IF EXISTS (SELECT name FROM sys.databases WHERE name =
'ManagementCompanyDB')
BEGIN
    ALTER DATABASE ManagementCompanyDB SET SINGLE_USER WITH ROLLBACK
IMMEDIATE;
    DROP DATABASE ManagementCompanyDB;
    PRINT 'База данных ManagementCompanyDB удалена';
END
GO
CREATE DATABASE ManagementCompanyDB;
GO

USE ManagementCompanyDB;
GO

PRINT 'База данных ManagementCompanyDB создана';
GO
CREATE TABLE Buildings (
    BuildingID INT IDENTITY(1,1) PRIMARY KEY,
    Address NVARCHAR(300) NOT NULL UNIQUE,
    Floors INT NOT NULL,
    ApartmentsCount INT NOT NULL,
    YearBuilt INT,
    TotalArea DECIMAL(10,2),
    ManagementStartDate DATE NOT NULL,
    Status NVARCHAR(20) DEFAULT 'Активен'
    CHECK (Status IN ('Активен', 'На реконструкции', 'Закрыт')),
    Notes NVARCHAR(500)
);
GO

CREATE TABLE Apartments (
    ApartmentID INT IDENTITY(1,1) PRIMARY KEY,
    BuildingID INT NOT NULL,
    ApartmentNumber INT NOT NULL,
    OwnerFullName NVARCHAR(150) NOT NULL,
    Phone NVARCHAR(20) NOT NULL,
    Area DECIMAL(8,2),
    RoomsCount INT,
    Status NVARCHAR(20) DEFAULT 'Заселена'
    CHECK (Status IN ('Заселена', 'Пустует', 'На ремонте',
'Продается')),
    FOREIGN KEY (BuildingID) REFERENCES Buildings(BuildingID) ON DELETE
CASCADE

```

```

);
GO

CREATE TABLE Employees (
    EmployeeID INT IDENTITY(1,1) PRIMARY KEY,
    LastName NVARCHAR(50) NOT NULL,
    FirstName NVARCHAR(50) NOT NULL,
    MiddleName NVARCHAR(50),
    Position NVARCHAR(50) NOT NULL,
    Department NVARCHAR(50),
    Phone NVARCHAR(20),
    Email NVARCHAR(100),
    HireDate DATE DEFAULT GETDATE(),
    IsActive BIT DEFAULT 1,
    Specialization NVARCHAR(200)
);
GO

CREATE TABLE RepairRequests (
    RequestID INT IDENTITY(1,1) PRIMARY KEY,
    RequestNumber NVARCHAR(20) NOT NULL UNIQUE,
    BuildingID INT NOT NULL,
    ApartmentID INT,
    ApplicantFullName NVARCHAR(150) NOT NULL,
    ApplicantPhone NVARCHAR(20) NOT NULL,
    ProblemDescription NVARCHAR(1000) NOT NULL,
    RequestDate DATETIME DEFAULT GETDATE(),
    Priority INT DEFAULT 3 CHECK (Priority BETWEEN 1 AND 5),
    Status NVARCHAR(20) DEFAULT 'Открыта'
    CHECK (Status IN ('Открыта', 'В работе', 'Заявка в работе',
'Заявка закрыта', 'Отменена')),
    AssignedToEmployeeID INT,
    EstimatedCompletionDate DATE,
    ActualCompletionDate DATETIME,
    CreatedByEmployeeID INT,
    Notes NVARCHAR(500),
    FOREIGN KEY (BuildingID) REFERENCES Buildings(BuildingID),
    FOREIGN KEY (ApartmentID) REFERENCES Apartments(ApartmentID),
    FOREIGN KEY (AssignedToEmployeeID) REFERENCES Employees(EmployeeID),
    FOREIGN KEY (CreatedByEmployeeID) REFERENCES Employees(EmployeeID)
);
GO

CREATE TABLE RequestHistory (
    HistoryID INT IDENTITY(1,1) PRIMARY KEY,
    RequestID INT NOT NULL,
    EmployeeID INT NOT NULL,
    ActionDate DATETIME DEFAULT GETDATE(),
    ActionType NVARCHAR(50) NOT NULL,
    Description NVARCHAR(500),
    OldStatus NVARCHAR(20),
    NewStatus NVARCHAR(20),
    FOREIGN KEY (RequestID) REFERENCES RepairRequests(RequestID) ON DELETE
CASCADE,
    FOREIGN KEY (EmployeeID) REFERENCES Employees(EmployeeID)
);
GO

CREATE INDEX IDX_Buildings_Address ON Buildings(Address);
CREATE INDEX IDX_Apartments_Building ON Apartments(BuildingID);
CREATE INDEX IDX_Apartments_Number ON Apartments(ApartmentNumber);
CREATE INDEX IDX_RepairRequests_Number ON RepairRequests(RequestNumber);
CREATE INDEX IDX_RepairRequests_Status ON RepairRequests(Status);
CREATE INDEX IDX_RepairRequests_Dates ON RepairRequests(RequestDate,

```

```

        ActualCompletionDate);
CREATE INDEX IDX_RepairRequests_Building ON RepairRequests(BuildingID);
CREATE INDEX IDX_RepairRequests_Employee ON RepairRequests(AssignedToEmployeeID);
CREATE INDEX IDX_RequestHistory_Request ON RequestHistory(RequestID);
GO
CREATE SEQUENCE RequestNumberSeq
    START WITH 1
    INCREMENT BY 1
    MINVALUE 1
    MAXVALUE 9999
    CYCLE;
GO
CREATE PROCEDURE dbo.GenerateRequestNumber
    @RequestNumber NVARCHAR(20) OUTPUT
AS
BEGIN
    DECLARE @Year INT = YEAR(GETDATE());
    DECLARE @Month INT = MONTH(GETDATE());
    DECLARE @NextNumber INT = NEXT VALUE FOR RequestNumberSeq;

    SET @RequestNumber = 'Z-' + CAST(@Year AS NVARCHAR(4)) +
        RIGHT('0' + CAST(@Month AS NVARCHAR(2)), 2) +
        '-' + RIGHT('0000' + CAST(@NextNumber AS
NVARCHAR(4)), 4);
END;
GO

CREATE TRIGGER TR_RepairRequests_LogHistory
ON RepairRequests
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;

    IF UPDATE(Status)
    BEGIN
        INSERT INTO RequestHistory (RequestID, EmployeeID, ActionType,
            OldStatus, NewStatus, Description)
        SELECT
            i.RequestID,
            ISNULL(i.AssignedToEmployeeID, i.CreatedByEmployeeID),
            'Изменение статуса',
            d.Status,
            i.Status,
            'Статус заявки изменен'
        FROM inserted i
        INNER JOIN deleted d ON i.RequestID = d.RequestID
        WHERE i.Status <> d.Status;
    END
END;
GO
PRINT 'Начало заполнения данных...';
GO
INSERT INTO Buildings (Address, Floors, ApartmentsCount, YearBuilt,
TotalArea, ManagementStartDate)
VALUES
    ('ул. 45 Параллель, 4/2, Ставрополь', 9, 52, 2001, 1978.7, '2015-04-
22'),
    ('ул. Васильева, 1, Ставрополь', 9, 144, 1983, 7950.4, '2015-04-22'),
    ('ул. Доваторцев, 66/2, Ставрополь', 9, 102, 1984, 3176.4, '2020-11-
01'),
    ('ул. Мира, 236, Ставрополь', 10, 62, 1991, 4423.1, '2007-11-04'),

```



```

('ул. Мира, 272, Ставрополь', 9, 88, 2006, 6204.7, '2015-04-22'),
('ул. Мира, 278, Ставрополь', 10, 40, 2008, 3294.1, '2019-08-01'),
('пл. Выставочная, 40, Светлоград', 5, 70, 1985, 2369.2, '2018-12-
01'),
('пл. Выставочная, 43, Светлоград', 5, 68, 1987, 3702.8, '2019-07-
01'),
('пл. Выставочная, 45, Светлоград', 4, 68, 1990, 3731.5, '2019-12-
01'),
('пл. Выставочная, 47, Светлоград', 5, 68, 1993, 4079.2, '2019-07-
01'),
('пл. Выставочная, 48, Светлоград', 5, 68, 1995, 3654.6, '2018-12-
01'),
('пл. Выставочная, 49, Светлоград', 5, 60, 1995, 2891.7, '2021-06-
01'),
('пл. Выставочная, 50, Светлоград', 5, 60, 1995, 4014.5, '2019-02-
01'),
('пл. Выставочная, 57, Светлоград', 3, 36, 2015, 2075.7, '2020-02-
01'),
('пл. Выставочная, 58, Светлоград', 5, 0, 2013, 3124.2, '2021-11-01'),
('ул. Бассейная, 82, Светлоград', 5, 48, 1988, 3255.3, '2021-03-01'),
('ул. Красная, 44а, Светлоград', 5, 60, 1983, 4317.4, '2022-03-01'),
('ул. Матросова, 179а, Светлоград', 4, 28, 1979, 879.4, '2022-02-18'),
('ул. Пушкина, 12, Светлоград', 5, 118, 1980, 5316.8, '2020-10-01'),
('ул. Пушкина, 3а, Светлоград', 5, 48, 1990, 1007.9, '2018-12-01'),
('ул. Ярмарочная, 21, Светлоград', 5, 58, 1985, 2586.6, '2021-01-01');
GO

PRINT 'Добавлено домов: ' + CAST(@@ROWCOUNT AS NVARCHAR(10));
GO

DECLARE @BuildingID INT;
SELECT @BuildingID = BuildingID FROM Buildings WHERE Address = 'ул.
Матросова, 179а, Светлоград';

INSERT INTO Apartments (BuildingID, ApartmentNumber, OwnerFullName, Phone)
VALUES
(@BuildingID, 1, 'Шевченко Ольга Викторовна', '+79180000001'),
(@BuildingID, 2, 'Мазалова Ирина Львовна', '+79185647218'),
(@BuildingID, 3, 'Семеняка Юрий Геннадьевич', '+79180000003'),
(@BuildingID, 4, 'Савельев Олег Иванович', '+79287815445'),
(@BuildingID, 5, 'Габиец Игорь Леонидович', '+79180000005'),
(@BuildingID, 6, 'Бунин Эдуард Михайлович', '+79180000006'),
(@BuildingID, 7, 'Бахшиев Павел Иннокентьевич', '+79180000007'),
(@BuildingID, 8, 'Байчорова Агата Рустамовна', '+89643324574'),
(@BuildingID, 9, 'Тюренкова Наталья Сергеевна', '+89629987214'),
(@BuildingID, 10, 'Александров Петр Константинович', '+79180000010'),
(@BuildingID, 11, 'Мазалова Ольга Николаевна', '+79180000011'),
(@BuildingID, 12, 'Лапшин Виктор Романович', '+79180000012'),
(@BuildingID, 13, 'Гусев Семен Петрович', '+89188601163'),
(@BuildingID, 14, 'Гладилина Вера Михайловна', '+79180000014'),
(@BuildingID, 15, 'Лукин Илья Федорович', '+89634568714'),
(@BuildingID, 16, 'Петров Станислав Игоревич', '+89189187845'),
(@BuildingID, 17, 'Филь Марина Федоровна', '+79180000017'),
(@BuildingID, 18, 'Михайлов Игорь Вадимович', '+79180000018'),
(@BuildingID, 19, 'Масюк Динара Викторовна', '+79180000019'),
(@BuildingID, 20, 'Мартыненко Александр Сергеевич', '+89183215428'),
(@BuildingID, 21, 'Устьянцева Анна Станиславовна', '+79180000021'),
(@BuildingID, 22, 'Антоненко Дмитрий Игоревич', '+79180000022'),
(@BuildingID, 23, 'Любашева Галина Аркадьевна', '+89625674581'),
(@BuildingID, 24, 'Захаряшев Денис Сергеевич', '+79180000024'),
(@BuildingID, 25, 'Третьяк Ярослава Викторовна', '+79180000025'),
(@BuildingID, 26, 'Бондарь Сергей Вадимович', '+79180000026'),
(@BuildingID, 27, 'Петраков Артем Сергеевич', '+79180000027'),

```

```

        (@BuildingID, 28, 'Вальке Рита Владимировна', '+79180000028');
GO

PRINT 'Добавлено квартир: ' + CAST(@@ROWCOUNT AS NVARCHAR(10));
GO

INSERT INTO Employees (LastName, FirstName, MiddleName, Position,
Department, Phone, Email)
VALUES
    ('Петров', 'Алексей', 'Иванович', 'Слесарь', 'Ремонтная служба',
'+79161234501', 'petrov@company.ru'),
    ('Иванова', 'Мария', 'Сергеевна', 'Электрик', 'Ремонтная служба',
'+79161234502', 'ivanova@company.ru'),
    ('Сидоров', 'Дмитрий', 'Александрович', 'Сантехник', 'Ремонтная
служба', '+79161234503', 'sidorov@company.ru'),
    ('Козлова', 'Ольга', 'Викторовна', 'Диспетчер', 'Приемная',
'+79161234504', 'kozlova@company.ru'),
    ('Васильев', 'Сергей', 'Петрович', 'Управляющий', 'Администрация',
'+79161234505', 'vasiliev@company.ru'),
    ('Николаев', 'Андрей', 'Михайлович', 'Мастер', 'Ремонтная служба',
'+79161234506', 'nikolaev@company.ru'),
    ('Федорова', 'Елена', 'Александровна', 'Бухгалтер', 'Финансовый
отдел', '+79161234507', 'fedorova@company.ru');
GO

PRINT 'Добавлено сотрудников: ' + CAST(@@ROWCOUNT AS NVARCHAR(10));
GO
DECLARE @BuildingID2 INT;
SELECT @BuildingID2 = BuildingID FROM Buildings WHERE Address = 'ул.
Матросова, 179а, Светлоград';
DECLARE @RequestNumber NVARCHAR(20);
DECLARE @Counter INT = 1;
DECLARE @ApartmentID INT;
DECLARE @OwnerFullName NVARCHAR(150);
DECLARE @Phone NVARCHAR(20);
DECLARE @ProblemDescription NVARCHAR(1000);
DECLARE @Priority INT;
DECLARE @Status NVARCHAR(20);
DECLARE @AssignedEmployeeID INT;
DECLARE @RequestDate DATETIME;
DECLARE apartment_cursor CURSOR FOR
SELECT TOP 10 ApartmentID, OwnerFullName, Phone
FROM Apartments
WHERE BuildingID = @BuildingID2
ORDER BY ApartmentNumber;

OPEN apartment_cursor;
FETCH NEXT FROM apartment_cursor INTO @ApartmentID, @OwnerFullName,
@Phone;

WHILE @@FETCH_STATUS = 0 AND @Counter <= 10
BEGIN
    EXEC dbo.GenerateRequestNumber @RequestNumber OUTPUT;

    SET @ProblemDescription = CASE @Counter
        WHEN 1 THEN 'Протекает кран на кухне'
        WHEN 2 THEN 'Не работает розетка в ванной'
        WHEN 3 THEN 'Забит слив в раковине'
        WHEN 4 THEN 'Трещина в стене на балконе'
        WHEN 5 THEN 'Не закрывается входная дверь'
        WHEN 6 THEN 'Протекает радиатор отопления'
        WHEN 7 THEN 'Сломан выключатель в коридоре'
        WHEN 8 THEN 'Запах из канализационного стояка'
    
```

```

        WHEN 9 THEN 'Отклеиваются обои в комнате'
        ELSE 'Общее обследование помещения'
    END;

    SET @Priority = CASE
        WHEN @Counter IN (1, 4) THEN 1
        WHEN @Counter IN (2, 5) THEN 2
        ELSE 3
    END;

    SET @Status = CASE
        WHEN @Counter <= 3 THEN 'Заявка закрыта'
        WHEN @Counter <= 6 THEN 'В работе'
        ELSE 'Открыта'
    END;

    SET @AssignedEmployeeID = CASE
        WHEN @Counter % 3 = 0 THEN 1
        WHEN @Counter % 3 = 1 THEN 2
        ELSE 3
    END;

    SET @RequestDate = DATEADD(day, -@Counter, GETDATE());

    INSERT INTO RepairRequests (
        RequestNumber,
        BuildingID,
        ApartmentID,
        ApplicantFullName,
        ApplicantPhone,
        ProblemDescription,
        Priority,
        Status,
        AssignedToEmployeeID,
        CreatedByEmployeeID,
        RequestDate
    )
    VALUES (
        @RequestNumber,
        @BuildingID2,
        @ApartmentID,
        @OwnerFullName,
        @Phone,
        @ProblemDescription,
        @Priority,
        @Status,
        @AssignedEmployeeID,
        4, -- CreatedByEmployeeID (диспетчер)
        @RequestDate
    );

    SET @Counter = @Counter + 1;
    FETCH NEXT FROM apartment_cursor INTO @ApartmentID, @OwnerFullName,
@Phone;
    END;

    CLOSE apartment_cursor;
    DEALLOCATE apartment_cursor;

    PRINT 'Добавлено заявок: ' + CAST(@Counter-1 AS NVARCHAR(10));
    GO
    UPDATE RepairRequests
    SET ActualCompletionDate = DATEADD(day, 2, RequestDate),

```

```
        EstimatedCompletionDate = DATEADD(day, 1, RequestDate)
WHERE Status = 'Заявка закрыта';
GO

PRINT 'Обновлено закрытых заявок: ' + CAST(@@ROWCOUNT AS NVARCHAR(10));
GO
UPDATE RepairRequests
SET EstimatedCompletionDate = DATEADD(day, 3, GETDATE())
WHERE Status = 'В работе';
GO

PRINT 'Обновлено заявок в работе: ' + CAST(@@ROWCOUNT AS NVARCHAR(10));
GO
```

РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

На рисунке 4 представлена Use-Case диаграмма. Для ее разработки использовалось приложение draw.io.

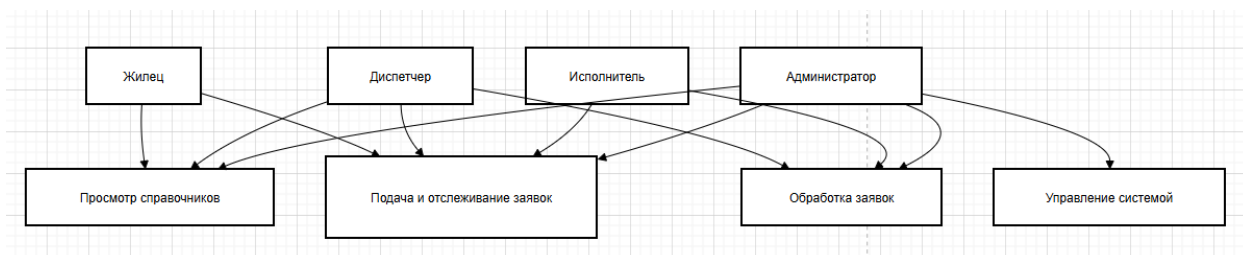


Рисунок 4 – Диаграмма USE CASE

На листинге 2 представлен код Use-Case диаграммы, отображающую принцип работы системы.

```
A1 [Жилец]
A2 [Диспетчер]
A3 [Исполнитель]
A4 [Администратор]
UC1 [Просмотр справочников]
UC2 [Подача и отслеживание заявок]
UC3 [Обработка заявок]
UC4 [Управление системой]
A1 A1 -->
--> UC1 UC2
A2 --> UC1
A2 --> UC2
A2 --> UC3
A3 --> UC2
A3 --> UC3
A4 --> UC1
A4 --> UC2
A4 --> UC3
A4 --> UC4
```

На рисунке 5 представлена блок-схема для обработки заявки на ремонт по ГОСТ 19.701-90.

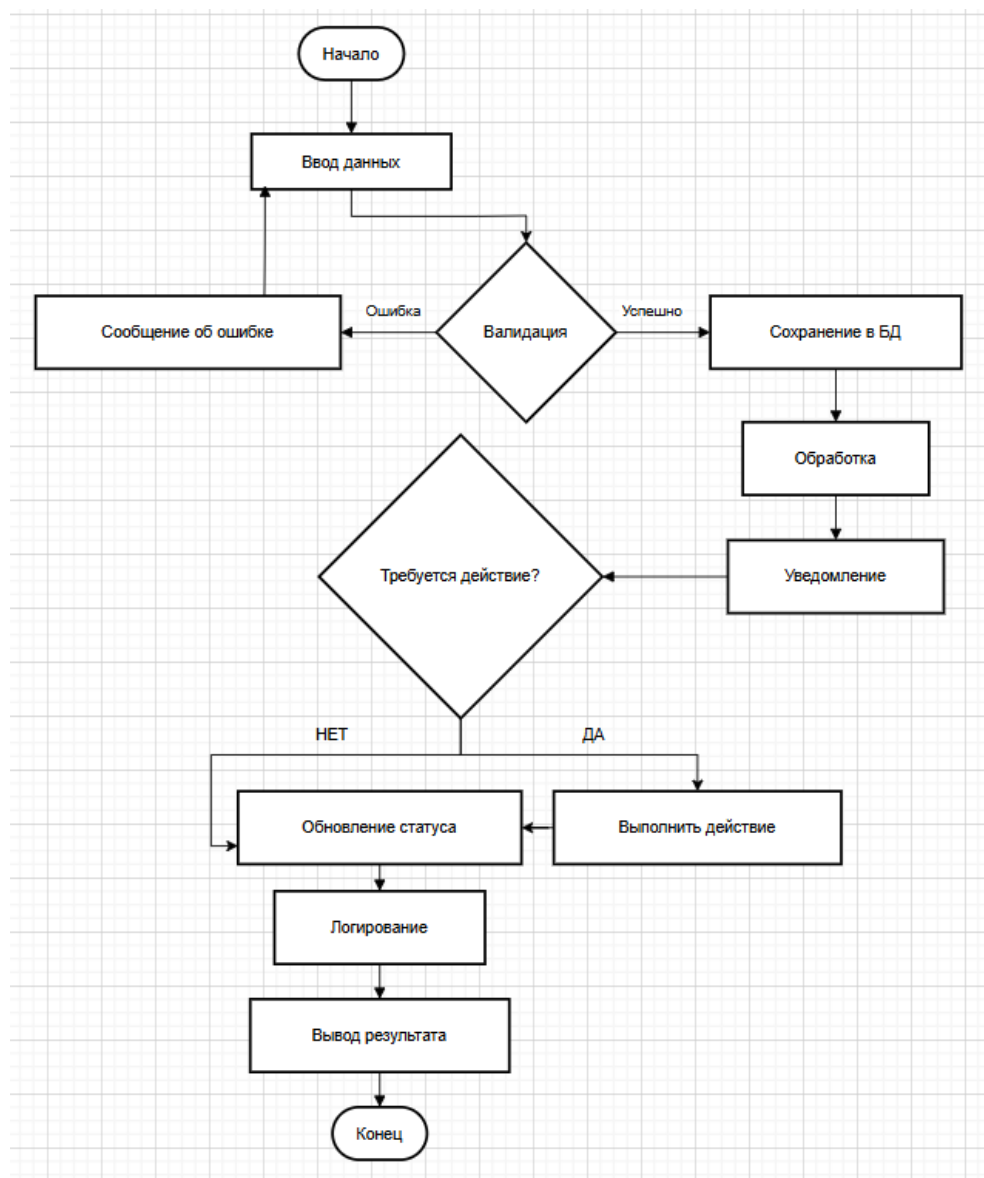


Рисунок 5 – Блок-схема для обработки заявки на ремонт

В таблицах 2-9 представлены тестовые сценарии.

Таблица 2 – Список адресов.

Поле	Значение
Тестовый пример	ТС-001
Приоритет тестирования	Высокий
Заголовок/название теста	Просмотр списка адресов домов
Краткое изложение теста	Проверка отображения списка адресов на главной форме
Этапы теста	1. Запустить приложение 2. Перейти на вкладку "Дома" 3. Прокрутить список
Тестовые данные	База данных с 21 записью домов

Ожидаемый результат	Отображается таблица со всеми адресами домов, корректная сортировка
Фактический результат	Отображается таблица со всеми адресами домов, корректная сортировка
Предварительное условие	База данных заполнена тестовыми данными
Постусловие	Список адресов отображается корректно

Таблица 3 – Создание заявки.

Поле	Значение
Тестовый пример	ТС-FUNC-001
Приоритет тестирования	Высокий
Заголовок/название теста	Подача новой заявки на ремонт
Краткое изложение теста	Проверка создания заявки с валидными данными
Этапы теста	1. Нажать "Новая заявка" 2. Выбрать адрес из списка 3. Ввести ФИО заявителя 4. Ввести телефон 5. Ввести описание проблемы 6. Нажать "Сохранить"
Тестовые данные	Адрес: ул. Матросова, 179а, Светлоград ФИО: Ильина Софья Павловна Телефон: +79161234567 Проблема: "Протекает кран"
Ожидаемый результат	Заявка сохраняется в БД, присваивается уникальный номер, статус "Открыта"
Фактический результат	Заявка сохраняется в БД, присваивается уникальный номер, статус "Открыта"
Предварительное условие	Пользователь в роли "Жилец"
Постусловие	Новая заявка отображается в общем списке

Таблица 4 – Назначение исполнителя заявке.

Поле	Значение
Тестовый пример	ТС-FUNC-002
Приоритет тестирования	Высокий
Заголовок/название теста	Назначение исполнителя заявке
Краткое изложение теста	Проверка функции назначения сотрудника на выполнение заявки

Этапы теста	1. Выбрать заявку со статусом "Открыта" 2. Нажать "Назначить" 3. Выбрать исполнителя из списка 4. Нажать "Подтвердить"
Тестовые данные	Заявка #Z-202403-0001 Исполнитель: Ильина Софья Павловна
Ожидаемый результат	Статус заявки меняется на "В работе", исполнитель назначается
Фактический результат	Статус заявки меняется на "В работе", исполнитель назначается
Предварительное условие	Пользователь в роли "Диспетчер"
Постусловие	В истории заявки появляется запись о назначении

Таблица 5 – Завершение выполнения заявки.

Поле	Значение
Тестовый пример	ТС-FUNC-003
Приоритет тестирования	Средний
Заголовок/название теста	Завершение выполнения заявки
Краткое изложение теста	Проверка входа с корректными данными
Этапы теста	1. Выбрать заявку со статусом "В работе" 2. Нажать "Завершить" 3. Ввести комментарий о выполненной работе 4. Нажать "Подтвердить"
Тестовые данные	Заявка #Z-202403-0001 Комментарий: "Кран заменен, течь устранена"
Ожидаемый результат	Статус заявки меняется на "Заявка закрыта", проставляется дата завершения
Фактический результат	Статус заявки меняется на "Заявка закрыта", проставляется дата завершения
Предварительное условие	Пользователь в роли "Исполнитель"
Постусловие	Заявка исключается из списка активных

Таблица 6 – Фильтрация по сотруднику.

Поле	Значение
Тестовый пример	ТС-REP-001
Приоритет тестирования	Средний
Заголовок/название теста	Просмотр истории заявок по сотруднику

Краткое изложение теста	Проверка формирования отчета по выполненным заявкам сотрудника
Этапы теста	1. Перейти в раздел "Отчеты" 2. Выбрать "По сотрудникам" 3. Выбрать сотрудника из списка 4. Указать период дат 5. Нажать "Сформировать"
Тестовые данные	Сотрудник: Гусев Артем Александрович Период: текущий месяц
Ожидаемый результат	Таблица с заявками, которые выполнял выбранный сотрудник
Фактический результат	Таблица с заявками, которые выполнял выбранный сотрудник
Предварительное условие	Пользователь в роли "Администратор"
Постусловие	Отчет сформирован и доступен для просмотра

Таблица 7 – Сохранение заявки с незаполненными полями.

Поле	Значение
Тестовый пример	ТС-ERR-001
Приоритет тестирования	Высокий
Заголовок/название теста	Попытка сохранения заявки с пустыми обязательными полями
Краткое изложение теста	Проверка валидации вводимых данных
Этапы теста	1. Нажать "Новая заявка" 2. Оставить поле "ФИО заявителя" пустым 3. Оставить поле "Телефон" пустым 4. Нажать "Сохранить"
Тестовые данные	Пустые поля: ФИО, Телефон
Ожидаемый результат	Сообщение об ошибке с указанием незаполненных полей, заявка не сохраняется
Фактический результат	Сообщение об ошибке с указанием незаполненных полей, заявка не сохраняется
Предварительное условие	Пользователь в любой роли
Постусловие	Форма остается открытой, курсор устанавливается в первое незаполненное поле

Таблица 8 – Проверка целостности данных.

Поле	Значение
Тестовый пример	ТС-DB-001
Приоритет тестирования	Средний

Заголовок/название теста	Проверка целостности данных при удалении дома
Краткое изложение теста	Проверка каскадного удаления связанных записей
Этапы теста	1. Выбрать дом из списка 2. Нажать "Удалить" 3. Подтвердить удаление
Тестовые данные	Дом с адресом: ул. Матросова, 179а, Светлоград
Ожидаемый результат	Дом удаляется вместе со всеми квартирами и заявками, связанными с ним
Фактический результат	Дом удаляется вместе со всеми квартирами и заявками, связанными с ним
Предварительное условие	Пользователь в роли "Администратор"
Постусловие	Записи удалены, ссылочная целостность не нарушена

Таблица 9 – Проверка системы безопасности.

Поле	Значение
Тестовый пример	TC-SEC-001
Приоритет тестирования	Высокий
Заголовок/название теста	Попытка доступа к функциям без авторизации
Краткое изложение теста	Проверка системы безопасности и разграничения прав доступа
Этапы теста	1. Запустить приложение без авторизации 2. Попытаться открыть форму добавления заявки 3. Попытаться открыть раздел администрирования
Тестовые данные	Отсутствие учетных данных
Ожидаемый результат	Открывается форма авторизации, доступ к функционалу ограничен
Фактический результат	Открывается форма авторизации, доступ к функционалу ограничен
Предварительное условие	Приложение запущено
Постусловие	Пользователь перенаправлен на экран входа

В листинге 3 представлена разметка XAML окна управления квартирами. Этот код описывает интерфейс окна, предназначенного для

просмотра информации о квартирах, сгруппированных по адресам домов. Пользователь может выбрать конкретный дом из выпадающего списка и увидеть в таблице все квартиры этого дома с данными о владельцах и характеристиками жилья.

Листинг 3 – фрагмент кода ApartmentsWindow.xaml

```
<Window x:Class="ManagementCompanyApp.ApartmentsWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Квартиры"
        Height="500"
        Width="800"
        WindowStartupLocation="CenterOwner"
        FontFamily="Segoe UI"
        Background="#FFFFFF"
        Icon="C:\Users\User\source\repos\ManagementCompanyApp\Res\icon.ico">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>

        <Border Grid.Row="0" Background="#67BA80" Padding="10"
            Margin="0,0,0,10" CornerRadius="5">
            <StackPanel>
                <TextBlock Text="Квартиры"
                    FontSize="20"
                    FontWeight="Bold"
                    Foreground="White"
                    HorizontalAlignment="Center"/>

                <WrapPanel HorizontalAlignment="Center" Margin="0,10,0,0">
                    <Label Content="Выберите дом:" Foreground="White"/>
                    <ComboBox x:Name="CbBuildings" Width="250"
                        DisplayMemberPath="Address"/>
                    <Button Content="Показать" Click="BtnShow_Click"
                        Margin="5"/>
                </WrapPanel>
            </StackPanel>
        </Border>
    </Grid>
```

```

        </Border>

        <DataGrid x:Name="DataGridApartments"
            Grid.Row="1"
            Background="#FFFFFF"
            AlternatingRowBackground="#F4E8D3"
            AutoGenerateColumns="False">
            <DataGrid.Columns>
                <DataGridTextColumn Header="% квартиры" Binding="{Binding
ApartmentNumber}" Width="80"/>
                <DataGridTextColumn Header="Владелец" Binding="{Binding
OwnerFullName}" Width="200"/>
                <DataGridTextColumn Header="Телефон" Binding="{Binding
Phone}" Width="120"/>
                <DataGridTextColumn Header="Площадь, м²" Binding="{Binding
Area, StringFormat=N1}" Width="90"/>
                <DataGridTextColumn Header="Комнат" Binding="{Binding
RoomsCount}" Width="70"/>
                <DataGridTextColumn Header="Статус" Binding="{Binding
Status}" Width="100"/>
            </DataGrid.Columns>
        </DataGrid>
    </Grid>
</Window>

```

В листинге 4 представлен код C# окна управления квартирами. Класс выполняет загрузку списка домов из таблицы Buildings при инициализации окна, а по нажатию кнопки "Показать" загружает и отображает в таблице все квартиры выбранного дома из таблицы Apartments.

Листинг 4 – фрагмент кода ApartmentsWindow.xaml.cs

```

using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows;
using System.Windows.Controls;

namespace ManagementCompanyApp
{
    public partial class ApartmentsWindow : Window

```

```

private string connectionString;

public ApartmentsWindow(string connectionString)
{
    InitializeComponent();
    this.connectionString = connectionString;
    LoadBuildings();
}

private void LoadBuildings()
{
    try
    {
        string query = "SELECT BuildingID, Address FROM Buildings
ORDER BY Address";

        using (SqlConnection connection = new
SqlConnection(connectionString))
            using (SqlCommand command = new SqlCommand(query,
connection))
                using (SqlDataAdapter adapter = new
SqlDataAdapter(command))
                {
                    DataTable dataTable = new DataTable();
                    adapter.Fill(dataTable);
                    CbBuildings.DisplayMemberPath = "Address";
                    CbBuildings.SelectedValuePath = "BuildingID";
                    CbBuildings.ItemsSource = dataTable.DefaultView;
                    if (dataTable.Rows.Count > 0)
                        CbBuildings.SelectedIndex = 0;
                }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка загрузки домов: {ex.Message}",
"Ошибка");
    }
}

private void BtnShow_Click(object sender, RoutedEventArgs e)
{
    if (CbBuildings.SelectedValue == null)
    {
        MessageBox.Show("Выберите дом", "Ошибка");
    }
}

```

```

        return;
    }
    try
    {
        int buildingId =
Convert.ToInt32(CbBuildings.SelectedValue);
        string query = @"SELECT ApartmentNumber, OwnerFullName,
Phone,
Area, RoomsCount, Status
FROM Apartments
WHERE BuildingID = @BuildingID
ORDER BY ApartmentNumber";

        using (SqlConnection connection = new
SqlConnection(connectionString))
        using (SqlCommand command = new SqlCommand(query,
connection))
        {
            command.Parameters.AddWithValue("@BuildingID",
buildingId);

            using (SqlDataAdapter adapter = new
SqlDataAdapter(command))
            {
                DataTable dataTable = new DataTable();
                adapter.Fill(dataTable);
                DataGridApartments.ItemsSource =
dataTable.DefaultView;
            }
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка загрузки квартир: {ex.Message}",
"Ошибка");
    }
}
}
}

```

В листинге 5 представлена разметка XAML окна жилого фонда. Этот код

описывает интерфейс окна, предназначенного для просмотра информации о домах. Пользователь видит таблицу с данными по всем многоквартирным домам: адрес, количество этажей и квартир, год постройки, общую площадь, дату начала управления и текущий статус.

Листинг 5 – фрагмент кода BuildingsWindow.xaml

```
<Window x:Class="ManagementCompanyApp.BuildingsWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Жилой фонд"
        Height="500"
        Width="900"
        WindowStartupLocation="CenterOwner"
        FontFamily="Segoe UI"
        Background="#FFFFFF"
        Icon="C:\Users\User\source\repos\ManagementCompanyApp\Res\icon.ico">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>

        <Border Grid.Row="0" Background="#67BA80" Padding="10"
            Margin="0,0,0,10" CornerRadius="5">
            <TextBlock Text="Жилой фонд"
                FontSize="20"
                FontWeight="Bold"
                Foreground="White"
                HorizontalAlignment="Center"/>
        </Border>

        <DataGrid x:Name="DataGridBuildings"
            Grid.Row="1"
            Background="#FFFFFF"
            AlternatingRowBackground="#F4E8D3"
            AutoGenerateColumns="False">
            <DataGrid.Columns>
                <DataGridTextColumn Header="Адрес" Binding="{Binding
Address}" Width="250"/>
            </DataGrid.Columns>
        </DataGrid>
    </Grid>
</Window>
```

```

        <DataGridTextColumn      Header="Этажей"      Binding="{Binding
Floors}" Width="60"/>
        <DataGridTextColumn      Header="Квартир"      Binding="{Binding
ApartmentsCount}" Width="70"/>
        <DataGridTextColumn      Header="Год постройки" Binding="{Binding
YearBuilt}" Width="100"/>
        <DataGridTextColumn      Header="Площадь, м²" Binding="{Binding
TotalArea, StringFormat=N1}" Width="90"/>
        <DataGridTextColumn      Header="Управление с" Binding="{Binding
ManagementStartDate, StringFormat=dd.MM.yyyy}" Width="110"/>
        <DataGridTextColumn      Header="Статус"      Binding="{Binding
Status}" Width="100"/>
    </DataGrid.Columns>
</DataGrid>
</Grid>
</Window>

```

В листинге 6 представлен код C# окна жилого фонда. Этот код содержит логику загрузки и отображения данных о домах из таблицы Buildings. При открытии окна автоматически выполняется SQL-запрос для получения

информации по всем многоквартирным домам, включая их основные характеристики и статусы. Данные выводятся в таблицу, реализована обработка исключений для информирования пользователя об ошибках подключения к базе данных.

Листинг 6 – фрагмент кода BuildingsWindow.xaml.cs

```

using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows;

namespace ManagementCompanyApp
{
    public partial class BuildingsWindow : Window
    {
        private string connectionString;

        public BuildingsWindow(string connectionString)
        {
            InitializeComponent();

```



```

        this.connectionString = connectionString;
        LoadBuildings();
    }

    private void LoadBuildings()
    {
        try
        {
            string query = @"SELECT Address, Floors, ApartmentsCount,
YearBuilt,
                                TotalArea, ManagementStartDate, Status
                                FROM Buildings ORDER BY Address";

            using (SqlConnection connection = new
SqlConnection(connectionString))
            using (SqlCommand command = new SqlCommand(query,
connection))
            using (SqlDataAdapter adapter = new SqlDataAdapter(command))
            {
                DataTable dataTable = new DataTable();
                adapter.Fill(dataTable);
                DataGridBuildings.ItemsSource = dataTable.DefaultView;
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка загрузки жилого фонда:
{ex.Message}", "Ошибка",
                MessageBoxButton.OK, MessageBoxImage.Error);
        }
    }
}

```

В листинге 7 представлена разметка XAML окна создания заявки на ремонт. Этот код описывает форму для добавления новой заявки с полями: выбор адреса дома, номер квартиры, ФИО заявителя, контактный телефон,

описание проблемы, ответственный исполнитель, статус и приоритет.
Обязательные поля отмечены звездочкой (*).

Листинг 7 – фрагмент кода CreateRequestWindow.xaml

```
<Window x:Class="ManagementCompanyApp.CreateRequestWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Создание заявки на ремонт"
        Height="550"
        Width="650"
        WindowStartupLocation="CenterOwner"
        FontFamily="Segoe UI"
        Background="#FFFFFF"
        Icon="C:\Users\User\source\repos\ManagementCompanyApp\Res\icon.ico">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <Border Grid.Row="0" Background="#67BA80" Padding="10"
            Margin="0,0,0,10" CornerRadius="5">
            <TextBlock Text="Создание новой заявки на ремонт"
                FontSize="18"
                FontWeight="Bold"
                Foreground="White"
                HorizontalAlignment="Center"/>
        </Border>

        <StackPanel Grid.Row="1" Margin="5">
            <Label Content="Адрес дома: *"/>
            <ComboBox x:Name="CbBuildings" DisplayMemberPath="Address"
                Height="30"/>

            <Label Content="Номер квартиры (опционально): *"/>
            <TextBox x:Name="TxtApartmentNumber" Height="30"/>

            <Label Content="ФИО заявителя: *"/>
            <TextBox x:Name="TxtApplicantName" Height="30"/>
        </StackPanel>
    </Grid>
</Window>
```

```

        <Label Content="Контактный телефон: *"/>
        <TextBox x:Name="TxtPhone" Height="30"/>

        <Label Content="Описание проблемы: *"/>
        <TextBox x:Name="TxtProblem" Height="100"
                VerticalScrollBarVisibility="Auto"
TextWrapping="Wrap"/>

        <Label Content="Ответственный исполнитель: "/>
        <ComboBox x:Name="CbEmployees" DisplayMemberPath="FullName"
Height="30"/>

        <Label Content="Статус: "/>
        <ComboBox x:Name="CbStatus" Height="30">
            <ComboBoxItem Content="Открыта" IsSelected="True"/>
            <ComboBoxItem Content="В работе"/>
            <ComboBoxItem Content="Заявка в работе"/>
            <ComboBoxItem Content="Заявка закрыта"/>
            <ComboBoxItem Content="Отменена"/>
        </ComboBox>

        <Label Content="Приоритет: "/>
        <ComboBox x:Name="CbPriority" Height="30">
            <ComboBoxItem Content="1 - Высокий"/>
            <ComboBoxItem Content="2" IsSelected="True"/>
            <ComboBoxItem Content="3"/>
            <ComboBoxItem Content="4"/>
            <ComboBoxItem Content="5 - Низкий"/>
        </ComboBox>
    </StackPanel>

    <StackPanel Grid.Row="2" Orientation="Horizontal"
HorizontalAlignment="Right">
        <Button Content="Отмена"
                Background="#67BA80"
                Foreground="White"
                Width="100"
                Height="35"
                Margin="5"
                Click="BtnCancel_Click"/>

```

```

        <Button Content="Сохранить"
            Background="#67BA80"
            Foreground="White"
            Width="100"
            Height="35"
            Margin="5"

            Click="BtnSave_Click"/>

    </StackPanel>
</Grid>
</Window>

```

В листинге 8 представлен код C# окна создания и редактирования заявки. Этот код содержит бизнес-логику для обработки операций создания новой заявки или редактирования существующей. Реализованы валидация вводимых данных, загрузка справочников домов и сотрудников, обработка номера квартиры с автоматическим созданием записи при необходимости. Код включает генерацию уникального номера заявки, обработку ошибок базы данных с понятными пользователю сообщениями и предупреждение о несохраненных изменениях при закрытии формы.

Листинг 8 – фрагмент кода CreateRequestWindow.xaml.cs

```

using System;
using System.Data;
using System.Data.SqlClient;
using System.Text.RegularExpressions;
using System.Windows;
using System.Windows.Controls;

namespace ManagementCompanyApp
{
    public partial class CreateRequestWindow : Window
    {
        private string connectionString;
        private int? editRequestId = null;

        public CreateRequestWindow(string connectionString, int? requestId = null)
        {
            InitializeComponent();

```

```

        this.connectionString = connectionString;
        this.editRequestId = requestId;
        LoadData();

        if (requestId.HasValue)
        {
            Title = "Редактирование заявки на ремонт";
            LoadRequestData(requestId.Value);
        }
        else
        {
            Title = "Создание заявки на ремонт";
        }

        // Добавляем обработчики событий для валидации
        TxtPhone.TextChanged += TxtPhone_TextChanged;
        TxtApplicantName.TextChanged += TxtApplicantName_TextChanged;
    }

    private void TxtPhone_TextChanged(object sender, TextChangedEventArgs
e)
    {
        string phone = TxtPhone.Text;
        if (!IsValidPhoneNumber(phone) && !string.IsNullOrEmpty(phone))
        {
            TxtPhone.ToolTip = "Телефон должен содержать только цифры,
начинаться с +7 или 8";
            TxtPhone.BorderBrush = System.Windows.Media.Brushes.Red;
        }
        else
        {
            TxtPhone.ToolTip = "Формат: +7XXXXXXXXXX или 8XXXXXXXXXX";
            TxtPhone.ClearValue(TextBox.BorderBrushProperty);
        }
    }

    private void TxtApplicantName_TextChanged(object sender,
TextChangedEventArgs e)
    {
        string name = TxtApplicantName.Text;
        if (name.Length > 150)
        {

```

```

        TxtApplicantName.ToolTip = "ФИО не должно превышать 150
СИМВОЛОВ";

        TxtApplicantName.BorderBrush =
System.Windows.Media.Brushes.Orange;
    }
    else
    {
        TxtApplicantName.ToolTip = "Введите полное ФИО заявителя";
        TxtApplicantName.ClearValue(TextBox.BorderBrushProperty);
    }
}

private bool IsValidPhoneNumber(string phone)
{
    if (string.IsNullOrEmpty(phone))
        return false;

    // Проверяем форматы: +7XXXXXXXXXX или 8XXXXXXXXXX
    Regex regex = new Regex(@"^(+7|8)\d{10}$");
    return regex.IsMatch(phone);
}

private void LoadData()
{
    try
    {
        // Загрузка домов
        string buildingsQuery = "SELECT BuildingID, Address FROM
Buildings ORDER BY Address";
        FillComboBox(CbBuildings, buildingsQuery, "Address",
"BuildingID");

        // Загрузка сотрудников
        string employeesQuery = @"SELECT EmployeeID,
                                LastName + ' ' + FirstName + ' ' +
ISNULL(MiddleName, '') AS FullName
                                FROM Employees WHERE IsActive = 1
ORDER BY LastName, FirstName";
        FillComboBox(CbEmployees, employeesQuery, "FullName",
"EmployeeID");
    }
    catch (Exception ex)

```

```

        {
            MessageBox.Show($"Ошибка загрузки данных: {ex.Message}\n\n" +
                            "Приложение продолжит работу, но некоторые функции могут быть недоступны.",
                            "Ошибка загрузки",
                            MessageBoxButtons.OK,
                            MessageBoxIcon.Warning);
        }
    }

    private void LoadRequestData(int requestId)
    {
        try
        {
            string query = @"SELECT r.BuildingID, r.ApartmentID,
r.ApplicantFullName,
r.ApplicantPhone, r.ProblemDescription,
r.Status,
r.Priority, r.AssignedToEmployeeID,
a.ApartmentNumber
FROM RepairRequests r
LEFT JOIN Apartments a ON r.ApartmentID =
a.ApartmentID
WHERE r.RequestID = @RequestID";

            using (SqlConnection connection = new
SqlConnection(connectionString))
            using (SqlCommand command = new SqlCommand(query,
connection))
            {
                command.Parameters.AddWithValue("@RequestID", requestId);
                connection.Open();

                using (SqlDataReader reader = command.ExecuteReader())
                {
                    if (reader.Read())
                    {
                        // Заполняем поля данными
                        if (reader["BuildingID"] != DBNull.Value)
                            CbBuildings.SelectedValue =
reader["BuildingID"];

```

```

        TxtApplicantName.Text =
reader["ApplicantFullName"].ToString();

        TxtPhone.Text =
reader["ApplicantPhone"].ToString();

        TxtProblem.Text =
reader["ProblemDescription"].ToString();

        // Заполняем номер квартиры
        if (reader["ApartmentNumber"] != DBNull.Value)
        {
            TxtApartmentNumber.Text =
reader["ApartmentNumber"].ToString();
        }

        // Устанавливаем статус
        string status = reader["Status"].ToString();
        foreach (ComboBoxItem item in CbStatus.Items)
        {
            if (item.Content.ToString() == status)
            {
                CbStatus.SelectedItem = item;
                break;
            }
        }

        // Устанавливаем приоритет
        if (reader["Priority"] != DBNull.Value)
        {
            int priority =
Convert.ToInt32(reader["Priority"]);

            if (priority >= 1 && priority <= 5)
                CbPriority.SelectedIndex = priority - 1;
        }

        // Устанавливаем исполнителя
        if (reader["AssignedToEmployeeID"] !=
DBNull.Value)
        {
            CbEmployees.SelectedValue =
reader["AssignedToEmployeeID"];
        }
    }
}

```



```

else
{
    MessageBox.Show("Заявка не найдена в базе
данных.\n\n" +
                    "Возможно, она была удалена другим
пользователем.",
                    "Заявка не найдена",
                    MessageBoxButton.OK,
                    MessageBoxImage.Warning);
    this.Close();
}
}
}
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка загрузки данных заявки:
{ex.Message}\n\n" +
                    "Проверьте подключение к базе данных и
повторите попытку.",
                    "Ошибка загрузки",
                    MessageBoxButton.OK,
                    MessageBoxImage.Error);
    this.Close();
}
}

private void FillComboBox(ComboBox comboBox, string query, string
displayMember, string valueMember)
{
    try
    {
        using (SqlConnection connection = new
SqlConnection(connectionString))
        using (SqlCommand command = new SqlCommand(query,
connection))
        using (SqlDataAdapter adapter = new SqlDataAdapter(command))
        {
            DataTable dataTable = new DataTable();
            adapter.Fill(dataTable);

            comboBox.DisplayMemberPath = displayMember;

```

```

        comboBox.SelectedValuePath = valueMember;
        comboBox.ItemsSource = dataTable.DefaultView;

        if (dataTable.Rows.Count > 0)
            comboBox.SelectedIndex = 0;
    }
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка загрузки данных в список:
{ex.Message}",
                    "Ошибка загрузки",
                    MessageBoxButton.OK,
                    MessageBoxImage.Warning);
}

private void BtnSave_Click(object sender, RoutedEventArgs e)
{
    try
    {
        // Валидация обязательных полей
        if (CbBuildings.SelectedValue == null)
        {
            MessageBox.Show("Выберите адрес дома из списка.\n\n" +
                            "Это обязательное поле для создания
заявки.",
                            "Не выбран адрес",
                            MessageBoxButton.OK,
                            MessageBoxImage.Warning);

            CbBuildings.Focus();
            return;
        }

        if (string.IsNullOrWhiteSpace(TxtApplicantName.Text))
        {
            MessageBox.Show("Введите ФИО заявителя.\n\n" +
                            "Это обязательное поле для создания
заявки.",
                            "Не указано ФИО",
                            MessageBoxButton.OK,
                            MessageBoxImage.Warning);
        }
    }
}

```

```

        TxtApplicantName.Focus();
        return;
    }

    if (string.IsNullOrEmpty(TxtPhone.Text))
    {
        MessageBox.Show("Введите контактный телефон.\n\n" +
            "Это обязательное поле для создания
заявки.",
            "Не указан телефон",
            MessageBoxButton.OK,
            MessageBoxImage.Warning);
        TxtPhone.Focus();
        return;
    }

    if (!IsValidPhoneNumber(TxtPhone.Text))
    {
        MessageBox.Show("Неверный формат телефона.\n\n" +
            "Телефон должен начинаться с +7 или 8 и
содержать 10 цифр.\n" +
            "Пример: +79161234567 или 89161234567",
            "Неверный формат телефона",
            MessageBoxButton.OK,
            MessageBoxImage.Warning);
        TxtPhone.Focus();
        TxtPhone.SelectAll();
        return;
    }

    if (string.IsNullOrEmpty(TxtProblem.Text))
    {
        MessageBox.Show("Введите описание проблемы.\n\n" +
            "Это обязательное поле для создания
заявки.\n" +
            "Опишите проблему подробно для быстрого
решения.",
            "Не указана проблема",
            MessageBoxButton.OK,
            MessageBoxImage.Warning);
        TxtProblem.Focus();
        return;
    }

```

```

    }

    // Проверка длины описания проблемы
    if (TxtProblem.Text.Length > 1000)
    {
        MessageBox.Show("Описание проблемы слишком длинное.\n\n"
+
        "Максимальная длина: 1000 символов.\n" +
        "Сейчас: " + TxtProblem.Text.Length + "
СИМВОЛОВ.",

        "Слишком длинное описание",
        MessageBoxButton.OK,
        MessageBoxImage.Warning);

        TxtProblem.Focus();
        return;
    }

    // Получаем данные
    int buildingId = Convert.ToInt32(CbBuildings.SelectedValue);
    string applicantName = TxtApplicantName.Text.Trim();
    string phone = TxtPhone.Text.Trim();
    string problem = TxtProblem.Text.Trim();
    string status =
((ComboBoxItem)CbStatus.SelectedItem).Content.ToString();

    // Определяем приоритет
    int priority = 3;
    if (CbPriority.SelectedItem is ComboBoxItem item)
    {
        string content = item.Content.ToString();
        if (content.StartsWith("1")) priority = 1;
        else if (content.StartsWith("2")) priority = 2;
        else if (content.StartsWith("3")) priority = 3;
        else if (content.StartsWith("4")) priority = 4;
        else if (content.StartsWith("5")) priority = 5;
    }

    // Получаем ID исполнителя
    int? employeeId = null;
    if (CbEmployees.SelectedValue != null)
    {
        employeeId = Convert.ToInt32(CbEmployees.SelectedValue);
    }

```

```

    }

    // Обрабатываем номер квартиры
    int? apartmentId = null;

    if (!string.IsNullOrWhiteSpace(TxtApartmentNumber.Text))
    {
        if (!int.TryParse(TxtApartmentNumber.Text, out int
aptNumber))
        {
            MessageBox.Show("Номер квартиры должен быть
числом.\n\n" +
                                "Пример: 25, 101, 42",
                                "Неверный номер квартиры",
                                MessageBoxButton.OK,
                                MessageBoxImage.Warning);
            TxtApartmentNumber.Focus();
            TxtApartmentNumber.SelectAll();
            return;
        }
        apartmentId = GetOrCreateApartmentId(buildingId,
aptNumber);
    }

    using (SqlConnection connection = new
SqlConnection(connectionString))
    {
        connection.Open();

        if (editRequestId.HasValue)
        {
            // Обновление существующей заявки
            UpdateRequest(connection, buildingId, apartmentId,
applicantName,
                                phone, problem, status, priority,
employeeId);
        }
        else
        {
            // Создание новой заявки
            CreateNewRequest(connection, buildingId, apartmentId,
applicantName,

```

```

phone, problem, status, priority,
employeeId);

        }
    }
}
catch (SqlException sqlEx)
{
    string errorMessage =
        GetUserFriendlyErrorMessage(sqlEx);
    MessageBox.Show(errorMessage,
        "Ошибка базы данных",
        MessageBoxButtons.OK,
        MessageBoxIcon.Error);
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка при сохранении заявки:
{ex.Message}\n\n" +
        "Проверьте введенные данные и повторите
попытку.",
        "Ошибка сохранения",
        MessageBoxButtons.OK,
        MessageBoxIcon.Error);
}
}

private void UpdateRequest(SqlConnection connection, int buildingId,
int? apartmentId,
string applicantName, string phone, string
problem,
string status, int priority, int?
employeeId)
{
    string updateQuery = @"UPDATE RepairRequests
        SET BuildingID = @BuildingID,
            ApartmentID = @ApartmentID,
            ApplicantFullName = @ApplicantFullName,
            ApplicantPhone = @ApplicantPhone,
            ProblemDescription =
@ProblemDescription,
            Status = @Status,
            Priority = @Priority,

```

```

AssignedToEmployeeID =
@AssignedToEmployeeID

WHERE RequestID = @RequestID";

using (SqlCommand command = new SqlCommand(updateQuery,
connection))
{
    command.Parameters.AddWithValue("@BuildingID", buildingId);
    command.Parameters.AddWithValue("@ApartmentID",
(object)apartmentId ?? DBNull.Value);
    command.Parameters.AddWithValue("@ApplicantFullName",
applicantName);
    command.Parameters.AddWithValue("@ApplicantPhone", phone);
    command.Parameters.AddWithValue("@ProblemDescription",
problem);
    command.Parameters.AddWithValue("@Status", status);
    command.Parameters.AddWithValue("@Priority", priority);
    command.Parameters.AddWithValue("@AssignedToEmployeeID",
(object)employeeId ?? DBNull.Value);
    command.Parameters.AddWithValue("@RequestID",
editRequestId.Value);

    int rowsAffected = command.ExecuteNonQuery();

    if (rowsAffected > 0)
    {
        MessageBox.Show("Заявка успешно обновлена!\n\n" +
            "Все изменения сохранены в системе.",
            "Успешное обновление",
            MessageBoxButton.OK,
            MessageBoxImage.Information);
        this.DialogResult = true;
        this.Close();
    }
    else
    {
        MessageBox.Show("Не удалось обновить заявку.\n\n" +
            "Возможно, заявка была удалена другим
пользователем.",
            "Ошибка обновления",
            MessageBoxButton.OK,
            MessageBoxImage.Error);
    }
}

```

```

        }
    }
}

private void CreateNewRequest(SqlConnection connection, int
buildingId, int? apartmentId,
                                string applicantName, string phone,
string problem,
                                string status, int priority, int?
employeeId)
{
    string requestNumber = GenerateRequestNumber();

    string insertQuery = @"INSERT INTO RepairRequests
                                (RequestNumber, BuildingID, ApartmentID,
                                ApplicantFullName, ApplicantPhone,
ProblemDescription,
                                Status, Priority, AssignedToEmployeeID,
CreatedByEmployeeID)
                                VALUES
                                (@RequestNumber, @BuildingID, @ApartmentID,
                                @ApplicantFullName, @ApplicantPhone,
@ProblemDescription,
                                @Status, @Priority, @AssignedToEmployeeID,
@CreatedByEmployeeID)";

    using (SqlCommand command = new SqlCommand(insertQuery,
connection))
    {
        command.Parameters.AddWithValue("@RequestNumber",
requestNumber);

        command.Parameters.AddWithValue("@BuildingID", buildingId);
        command.Parameters.AddWithValue("@ApartmentID",
(object)apartmentId ?? DBNull.Value);
        command.Parameters.AddWithValue("@ApplicantFullName",
applicantName);

        command.Parameters.AddWithValue("@ApplicantPhone", phone);
        command.Parameters.AddWithValue("@ProblemDescription",
problem);

        command.Parameters.AddWithValue("@Status", status);
        command.Parameters.AddWithValue("@Priority", priority);
        command.Parameters.AddWithValue("@AssignedToEmployeeID",
(object)employeeId ?? DBNull.Value);
    }
}

```



```

        command.Parameters.AddWithValue("@CreatedByEmployeeID", 4);

// ID диспетчера

        int rowsAffected = command.ExecuteNonQuery();

        if (rowsAffected > 0)
        {
            MessageBox.Show($"Заявка успешно создана!\n\n" +
                            $"Номер заявки: {requestNumber}\n" +
                            $"Статус: {status}\n" +
                            $"Приоритет: {priority}\n\n" +
                            $"Заявка добавлена в систему и доступна
для просмотра.",
                            "Успешное создание",
                            MessageBoxButtons.OK,
                            MessageBoxIcon.Information);

            this.DialogResult = true;
            this.Close();
        }
        else
        {
            MessageBox.Show("Не удалось создать заявку.\n\n" +
                            "Проверьте подключение к базе данных и
повторите попытку.",
                            "Ошибка создания",
                            MessageBoxButtons.OK,
                            MessageBoxIcon.Error);
        }
    }

    private int? GetOrCreateApartmentId(int buildingId, int
apartmentNumber)
    {
        try
        {
            // Сначала проверяем, существует ли уже квартира с таким
номером

            string checkQuery = @"SELECT ApartmentID FROM Apartments
                                WHERE BuildingID = @BuildingID AND
ApartmentNumber = @ApartmentNumber";

```

```

        using (SqlCommand command = new SqlCommand(checkQuery, new
SqlConnection(connectionString)))
        {
            command.Connection.Open();
            command.Parameters.AddWithValue("@BuildingID",
                buildingId);
            command.Parameters.AddWithValue("@ApartmentNumber",
                apartmentNumber);

            object result = command.ExecuteScalar();

            if (result != null)
            {
                return Convert.ToInt32(result);
            }
            else
            {
                // Создаем новую запись о квартире с неизвестным
                // владельцем
                string insertQuery = @"INSERT INTO Apartments
                (BuildingID, ApartmentNumber,
                OwnerFullName, Phone, Status)
                VALUES
                (@BuildingID, @ApartmentNumber,
                'Неизвестно', 'Не указан', 'Пустует');
                SELECT SCOPE_IDENTITY();"

                using (SqlCommand insertCommand = new
                SqlCommand(insertQuery, command.Connection))
                {
                    insertCommand.Parameters.AddWithValue("@BuildingID", buildingId);
                    insertCommand.Parameters.AddWithValue("@ApartmentNumber", apartmentNumber);

                    int newApartmentId =
                    Convert.ToInt32(insertCommand.ExecuteScalar());
                    return newApartmentId;
                }
            }
        }
    }
}

```

```

        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка при обработке номера квартиры:
{ex.Message}\n\n" +
                            "Квартира не будет привязана к заявке, но
заявка будет создана.",
                            "Ошибка обработки квартиры",
                            MessageBoxButton.OK,
                            MessageBoxImage.Warning);

            return null;
        }
    }

    private string GenerateRequestNumber()
    {
        return $"Z-{DateTime.Now:yyyyMMdd}-{new Random().Next(1000,
9999)}";
    }

    private string GetUserFriendlyErrorMessage(SqlException sqlEx)
    {
        switch (sqlEx.Number)
        {
            case 547:
                return "Ошибка ссылочной целостности.\n\n" +
                    "Выбранные данные не существуют или были удалены
из базы данных.\n\n" +
                    "Проверьте выбранный дом или исполнителя.";
            case 2627:
                return "Заявка с таким номером уже существует.\n\n" +
                    "Повторите попытку создания заявки.";
            case 515:
                return "Не все обязательные поля заполнены.\n\n" +
                    "Заполните все обязательные поля (помечены
звездочкой *).";
            case 208:
                return "Ошибка структуры базы данных.\n\n" +
                    "Таблица не найдена. Проверьте подключение к базе
данных.";
            default:
                return $"Ошибка базы данных: {sqlEx.Message}\n\n" +
                    $"Код ошибки: {sqlEx.Number}";
        }
    }

```

```

    }
}

private void BtnCancel_Click(object sender, RoutedEventArgs e)
{
    if (IsDataChanged())
    {
        var result = MessageBox.Show("Есть несохраненные
изменения.\n\n" +
                                     "Вы уверены, что хотите закрыть
форму без сохранения?",
                                     "Подтверждение закрытия",
                                     MessageBoxButton.YesNo,
                                     MessageBoxImage.Question);
        if (result == MessageBoxResult.No)
            return;
    }
    this.DialogResult = false;
    this.Close();
}

private bool IsDataChanged()
{
    return !string.IsNullOrEmpty(TxtApplicantName.Text) ||
           !string.IsNullOrEmpty(TxtPhone.Text) ||
           !string.IsNullOrEmpty(TxtProblem.Text);
}

private void TxtApartmentNumber_PreviewTextInput(object sender,
System.Windows.Input.TextCompositionEventArgs e)
{
    foreach (char c in e.Text)
    {
        if (!char.IsDigit(c))
        {
            e.Handled = true;
            break;
        }
    }
}
}

```

В листинге 9 представлена разметка XAML окна просмотра сотрудников. Этот код описывает интерфейс для отображения информации о персонале управляющей компании. Пользователь видит таблицу с данными сотрудников: фамилия, имя, отчество, должность, отдел, контактный телефон и адрес электронной почты.

Листинг 9 – фрагмент кода EmployeesWindow.xaml

```
<Window x:Class="ManagementCompanyApp.EmployeesWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Сотрудники"
        Height="500"
        Width="800"
        WindowStartupLocation="CenterOwner"
        FontFamily="Segoe UI"
        Background="#FFFFFF"
        Icon="C:\Users\User\source\repos\ManagementCompanyApp\Res\icon.ico">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>

        <Border Grid.Row="0" Background="#67BA80" Padding="10"
            Margin="0,0,0,10" CornerRadius="5">
            <TextBlock Text="Сотрудники"
                FontSize="20"
                FontWeight="Bold"
                Foreground="White"
                HorizontalAlignment="Center"/>
        </Border>

        <DataGrid x:Name="DataGridEmployees"
            Grid.Row="1"
            Background="#FFFFFF"
            AlternatingRowBackground="#F4E8D3"
            AutoGenerateColumns="False">
            <DataGrid.Columns>
```

```

        <DataGridTextColumn      Header="Фамилия"      Binding="{Binding
LastName}" Width="120"/>
        <DataGridTextColumn      Header="Имя"          Binding="{Binding
FirstName}" Width="120"/>
        <DataGridTextColumn      Header="Отчество"      Binding="{Binding
MiddleName}" Width="120"/>
        <DataGridTextColumn      Header="Должность"     Binding="{Binding
Position}" Width="150"/>
        <DataGridTextColumn      Header="Отдел"         Binding="{Binding
Department}" Width="120"/>
        <DataGridTextColumn      Header="Телефон"       Binding="{Binding
Phone}" Width="120"/>
        <DataGridTextColumn      Header="Email"         Binding="{Binding Email}"
Width="200"/>
    </DataGrid.Columns>
</DataGrid>
</Grid>
</Window>

```

В листинге 10 представлен код C# окна просмотра сотрудников. Этот код содержит логику загрузки данных о сотрудниках из таблицы Employees. При открытии окна выполняется SQL-запрос для получения информации только об активных сотрудниках (IsActive = 1), включая их личные данные, должности и контактную информацию. Данные сортируются по фамилии и имени для удобного просмотра, реализована обработка исключений для информирования пользователя об ошибках подключения.

Листинг 10 – фрагмент кода EmployeesWindow.xaml.cs

```

using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows;

namespace ManagementCompanyApp
{
    public partial class EmployeesWindow : Window
    {
        private string connectionString;

```

```

        public EmployeesWindow(string connectionString)
        {
            InitializeComponent();
            this.connectionString = connectionString;
            LoadEmployees();
        }

        private void LoadEmployees()
        {
            try
            {
                string query = @"SELECT LastName, FirstName, MiddleName,
                                   Position, Department, Phone, Email
                                   FROM Employees WHERE IsActive = 1
                                   ORDER BY LastName, FirstName";

                using (SqlConnection connection = new
SqlConnection(connectionString))
                    using (SqlCommand command = new SqlCommand(query,
connection))
                        using (SqlDataAdapter adapter = new
SqlDataAdapter(command))
                            {
                                DataTable dataTable = new DataTable();
                                adapter.Fill(dataTable);
                                DataGridEmployees.ItemsSource = dataTable.DefaultView;
                            }
            }
            catch (Exception ex)
            {
                MessageBox.Show($"Ошибка загрузки сотрудников:
{ex.Message}", "Ошибка");
            }
        }
    }
}

```

В листинге 11 представлена разметка XAML главного окна приложения. Этот код описывает основной интерфейс системы с меню навигации слева и контейнером для отображения контента справа. Меню включает кнопки для перехода к разделам: заявки на ремонт, жилой фонд, квартиры, сотрудники,

история заявок, статистика, а также кнопки создания новой заявки и возврата назад. Внизу панели отображается статус подключения к базе данных с кнопкой для тестирования соединения.

Листинг 11 – фрагмент кода EmployeesWindow.xaml

```
<Window x:Class="ManagementCompanyApp.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Управляющая компания - Учет заявок"
        Height="700" Width="1200"
        WindowStartupLocation="CenterScreen"
        FontFamily="Segoe UI"
        Background="#FFFFFF"

Icon="C:\Users\User\source\repos\ManagementCompanyApp\Res\icon.ico">

    <Window.Resources>
        <Style TargetType="Button">
            <Setter Property="Background" Value="#67BA80"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="FontSize" Value="14"/>
            <Setter Property="Height" Value="35"/>
            <Setter Property="Margin" Value="5"/>
            <Setter Property="Cursor" Value="Hand"/>
            <Setter Property="BorderBrush" Value="#67BA80"/>
            <Setter Property="BorderThickness" Value="1"/>
        </Style>

        <Style TargetType="DataGrid">
            <Setter Property="Background" Value="#FFFFFF"/>
            <Setter Property="AlternatingRowBackground" Value="#F4E8D3"/>
            <Setter Property="AutoGenerateColumns" Value="False"/>
            <Setter Property="CanUserAddRows" Value="False"/>
            <Setter Property="CanUserDeleteRows" Value="False"/>
            <Setter Property="BorderBrush" Value="#67BA80"/>
            <Setter Property="BorderThickness" Value="1"/>
        </Style>

        <Style TargetType="Label">
            <Setter Property="Foreground" Value="#333333"/>
            <Setter Property="FontSize" Value="14"/>
            <Setter Property="Margin" Value="5"/>
        </Style>
    </Window.Resources>
</Window>
```



```

        <Style TargetType="TextBox">
            <Setter Property="Background" Value="White"/>
            <Setter Property="BorderBrush" Value="#67BA80"/>
            <Setter Property="BorderThickness" Value="1"/>
            <Setter Property="FontSize" Value="14"/>
            <Setter Property="Margin" Value="5"/>
            <Setter Property="Height" Value="30"/>
        </Style>

        <Style TargetType="ComboBox">
            <Setter Property="Background" Value="White"/>
            <Setter Property="BorderBrush" Value="#67BA80"/>
            <Setter Property="BorderThickness" Value="1"/>
            <Setter Property="FontSize" Value="14"/>
            <Setter Property="Margin" Value="5"/>
            <Setter Property="Height" Value="30"/>
        </Style>
    </Window.Resources>

    <Grid>
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="220"/>
            <ColumnDefinition Width="*"/>
        </Grid.ColumnDefinitions>

        <Border Grid.Column="0" Background="#F4E8D3" BorderBrush="#67BA80"
BorderThickness="0,0,2,0">
            <StackPanel Margin="10">
                <Image
Source="C:\Users\User\source\repos\ManagementCompanyApp\Res\logo.png"
Height="60" Margin="0,0,0,20" Stretch="Uniform"/>

                <Label Content="Меню" FontSize="18" FontWeight="Bold"
HorizontalAlignment="Center" Margin="0,0,0,20"/>

                <Button x:Name="BtnRequests" Content="Заявки на ремонт"
Click="BtnRequests_Click"/>
                <Button x:Name="BtnBuildings" Content="Жилой фонд"
Click="BtnBuildings_Click"/>
                <Button x:Name="BtnApartments" Content="Квартиры"
Click="BtnApartments_Click"/>
            </StackPanel>
        </Border>
    </Grid>

```

```

        <Button      x:Name="BtnEmployees"      Content="Сотрудники"
Click="BtnEmployees_Click"/>

        <Button      x:Name="BtnHistory"      Content="История заявок"
Click="BtnHistory_Click"/>

        <Button      x:Name="BtnStatistics"      Content="Статистика"
Click="BtnStatistics_Click"/>

        <Separator Margin="0,20,0,10"/>

        <Button x:Name="BtnAddRequest" Content="+ Новая заявка"
Background="#67BA80" Click="BtnAddRequest_Click"/>
        <Button      x:Name="BtnBack"      Content="← Назад"
Click="BtnBack_Click"
        IsEnabled="False"/>

        <Separator Margin="0,20,0,10"/>

        <StackPanel>
            <Label Content="Статус подключения:"/>
            <Border      x:Name="ConnectionStatus"      CornerRadius="5"
Margin="5" Height="25">
                <TextBlock x:Name="TxtConnectionStatus" Text="He
подключено"
                HorizontalAlignment="Center"
VerticalAlignment="Center"/>
            </Border>
            <Button      x:Name="BtnTestConnection"      Content="Тест
подключения"
                Click="BtnTestConnection_Click" Margin="5"/>
        </StackPanel>
    </StackPanel>
</Border>

    <Frame      x:Name="MainFrame"      Grid.Column="1"
NavigationUIVisibility="Hidden"/>

</Grid>
</Window>

```

В листинге 12 представлен код C# главного окна приложения. Этот код содержит логику навигации между окнами системы и управления подключением к базе данных. Реализована функция проверки соединения с

базой данных при запуске приложения, обработка нажатий на кнопки меню для открытия соответствующих окон (заявки, жилой фонд, квартиры, сотрудники, история, статистика), а также логика для кнопок "Новая заявка" и "Назад". Код включает обработку исключений при открытии окон и отображение понятных сообщений об ошибках пользователю.

Листинг 12 – фрагмент кода EmployeesWindow.xaml.cs

```
using System;
using System.Data.SqlClient;
using System.Windows;
using System.Windows.Media;

namespace ManagementCompanyApp
{
    public partial class MainWindow : Window
    {
        private string connectionString = @"Data
Source=DESKTOR\SQLEXPRESSSS;Initial Catalog=ManagementCompanyDB;Integrated
Security=True";

        public MainWindow()
        {
            InitializeComponent();
            CheckDatabaseConnection();
            UpdateBackButtonState();
        }

        private void CheckDatabaseConnection()
        {
            try
            {
                using (SqlConnection connection = new
SqlConnection(connectionString))
                {
                    connection.Open();
                    UpdateConnectionStatus(true);
                }
            }
            catch (Exception ex)
            {
                UpdateConnectionStatus(false);
            }
        }
    }
}
```

```

        MessageBox.Show($"Ошибка подключения к базе данных:
{ex.Message}\n\n" +
        "Проверьте:\n" +
        "1. Запущен ли SQL Server\n" +
        "2. Правильность строки подключения\n" +
        "3. Существует ли база данных
ManagementCompanyDB",
        "Ошибка подключения",
        MessageBoxButton.OK,
        MessageBoxImage.Error);
    }
}

private void UpdateConnectionStatus(bool isConnected)
{
    if (isConnected)
    {
        ConnectionStatus.Background = Brushes.LightGreen;
        TxtConnectionStatus.Text = "Подключено";
    }
    else
    {
        ConnectionStatus.Background = Brushes.LightCoral;
        TxtConnectionStatus.Text = "Не подключено";
    }
}

private void UpdateBackButtonState()
{
    BtnBack.IsEnabled = (OwnedWindows != null &&
OwnedWindows.Count > 0);
}

private void BtnRequests_Click(object sender, RoutedEventArgs e)
{
    try
    {
        var window = new RequestsWindow(connectionString);
        window.Owner = this;
        window.Closed += (s, args) => UpdateBackButtonState();
        window.Show();
        UpdateBackButtonState();
    }
}

```

```

    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка открытия окна заявок:
{ex.Message}",
            "Ошибка открытия",
            MessageBoxButton.OK,
            MessageBoxImage.Error);
    }
}

private void BtnBuildings_Click(object sender, RoutedEventArgs e)
{
    try
    {
        var window = new BuildingsWindow(connectionString);
        window.Owner = this;
        window.Closed += (s, args) => UpdateBackButtonState();
        window.Show();
        UpdateBackButtonState();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка открытия окна жилого фонда:
{ex.Message}",
            "Ошибка открытия",
            MessageBoxButton.OK,
            MessageBoxImage.Error);
    }
}

private void BtnApartments_Click(object sender, RoutedEventArgs e)
{
    try
    {
        var window = new ApartmentsWindow(connectionString);
        window.Owner = this;
        window.Closed += (s, args) => UpdateBackButtonState();
        window.Show();
        UpdateBackButtonState();
    }
    catch (Exception ex)

```

```

        {
            MessageBox.Show($"Ошибка открытия окна квартир:
{ex.Message}",
                "Ошибка открытия",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }
    }

    private void BtnEmployees_Click(object sender, RoutedEventArgs e)
    {
        try
        {
            var window = new EmployeesWindow(connectionString);
            window.Owner = this;
            window.Closed += (s, args) => UpdateBackButtonState();
            window.Show();
            UpdateBackButtonState();
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Ошибка открытия окна сотрудников:
{ex.Message}",
                "Ошибка открытия",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }
    }

    private void BtnHistory_Click(object sender, RoutedEventArgs e)
    {
        try
        {
            var window = new RequestHistoryWindow(connectionString);
            window.Owner = this;
            window.Closed += (s, args) => UpdateBackButtonState();
            window.Show();
            UpdateBackButtonState();
        }
        catch (Exception ex)
        {

```

```

        MessageBox.Show($"Ошибка открытия окна истории:
{ex.Message}",
        "Ошибка открытия",
        MessageBoxButton.OK,
        MessageBoxImage.Error);
    }
}

private void BtnStatistics_Click(object sender, RoutedEventArgs e)
{
    try
    {
        var window = new StatisticsWindow(connectionString);
        window.Owner = this;
        window.Closed += (s, args) => UpdateBackButtonState();
        window.Show();
        UpdateBackButtonState();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка открытия окна статистики:
{ex.Message}",
        "Ошибка открытия",
        MessageBoxButton.OK,
        MessageBoxImage.Error);
    }
}

private void BtnAddRequest_Click(object sender, RoutedEventArgs e)
{
    try
    {
        var window = new CreateRequestWindow(connectionString);
        window.Owner = this;
        window.Closed += (s, args) => UpdateBackButtonState();
        bool? result = window.ShowDialog();

        if (result == true)
        {
            MessageBox.Show("Заявка успешно создана!\n\n" +
                "Новая заявка добавлена в систему и
доступна для просмотра.",

```

```

                                "Успешное создание",
                                MessageBoxButton.OK,
                                MessageBoxImage.Information);
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка создания заявки: {ex.Message}",
            "Ошибка создания",
            MessageBoxButton.OK,
            MessageBoxImage.Error);
    }
}

private void BtnBack_Click(object sender, RoutedEventArgs e)
{
    try
    {
        bool closedWindow = false;
        foreach (Window window in OwnedWindows)
        {
            if (window.IsActive)
            {
                window.Close();
                closedWindow = true;
                break;
            }
        }

        if (!closedWindow && OwnedWindows.Count > 0)
        {
            OwnedWindows[0].Close();
        }

        UpdateBackButtonState();
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка при закрытии окна: {ex.Message}",
            "Ошибка закрытия",
            MessageBoxButton.OK,
            MessageBoxImage.Error);
    }
}

```



```

    }
}

private void BtnTestConnection_Click(object sender,
RoutedEventArgs e)
{
    CheckDatabaseConnection();
    if (ConnectionStatus.Background == Brushes.LightGreen)
    {
        MessageBox.Show("Подключение к базе данных успешно
установлено!\n\n" +
                        "Сервер: DESKTOP\\SQLEXPRESSSS\n" +
                        "База данных: ManagementCompanyDB",
                        "Подключение успешно",
                        MessageBoxButton.OK,
                        MessageBoxImage.Information);
    }
    else
    {
        MessageBox.Show("Не удалось подключиться к базе
данных.\n\n" +
                        "Возможные причины:\n" +
                        "1. SQL Server не запущен\n" +
                        "2. Неправильное имя сервера\n" +
                        "3. Отсутствует база данных\n\n" +
                        "Проверьте настройки подключения и
запустите тест снова.",
                        "Ошибка подключения",
                        MessageBoxButton.OK,
                        MessageBoxImage.Error);
    }
}
}
}

```

В листинге 13 представлена разметка XAML окна истории заявок. Этот код описывает интерфейс для просмотра и фильтрации истории выполнения заявок. Пользователь может выбирать фильтр по сотруднику или адресу через выпадающие списки, после чего отображается таблица с данными: дата действия, номер заявки, адрес, тип действия, статус, сотрудник и описание.

Листинг 13 – фрагмент кода RequestHistoryWindow.xaml

```

<Window x:Class="ManagementCompanyApp.RequestHistoryWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="История заявок"
        Height="500"
        Width="900"
        WindowStartupLocation="CenterOwner"
        FontFamily="Segoe UI"
        Background="#FFFFFF"

Icon="C:\Users\User\source\repos\ManagementCompanyApp\Res\icon.ico">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>

        <Border Grid.Row="0" Background="#67BA80" Padding="10"
Margin="0,0,0,10" CornerRadius="5">
            <StackPanel>
                <TextBlock Text="История выполнения заявок"
                    FontSize="20"
                    FontWeight="Bold"
                    Foreground="White"
                    HorizontalAlignment="Center"/>

                <WrapPanel HorizontalAlignment="Center" Margin="0,10,0,0">
                    <Label Content="Фильтровать по:" Foreground="White"/>
                    <ComboBox x:Name="CbFilterType" Width="150"
Margin="5">
                        <ComboBoxItem Content="Сотруднику"
IsSelected="True"/>
                        <ComboBoxItem Content="Адресу"/>
                    </ComboBox>

                    <ComboBox x:Name="CbFilterValue" Width="200"
Margin="5"/>

                    <Button Content="Показать"
                        Background="White"
                        Foreground="#67BA80"

```

```

        Width="100"
        Height="30"
        Margin="5"
        Click="BtnShow_Click"/>
    </WrapPanel>
</StackPanel>
</Border>

<DataGrid x:Name="DataGridHistory"
    Grid.Row="1"
    Background="#FFFFFF"
    AlternatingRowBackground="#F4E8D3"
    AutoGenerateColumns="False">
    <DataGrid.Columns>
        <DataGridTextColumn Header="Дата" Binding="{Binding
ActionDate, StringFormat=dd.MM.yyyy HH:mm}" Width="120"/>
        <DataGridTextColumn Header="Номер заявки"
Binding="{Binding RequestNumber}" Width="100"/>
        <DataGridTextColumn Header="Адрес" Binding="{Binding
Address}" Width="200"/>
        <DataGridTextColumn Header="Тип действия"
Binding="{Binding ActionType}" Width="120"/>
        <DataGridTextColumn Header="Статус" Binding="{Binding
NewStatus}" Width="100"/>
        <DataGridTextColumn Header="Сотрудник" Binding="{Binding
EmployeeName}" Width="150"/>
        <DataGridTextColumn Header="Описание" Binding="{Binding
Description}" Width="*"/>
    </DataGrid.Columns>
</DataGrid>
</Grid>
</Window>

```

В листинге 14 представлен код C# окна истории заявок. Этот код содержит логику для фильтрации и отображения истории выполнения заявок. Реализована динамическая загрузка данных фильтра (сотрудники или адреса) в зависимости от выбранного типа фильтра, выполнение SQL-запросов для получения истории по выбранному критерию и отображение результатов в таблице. Код включает обработку ошибок подключения к базе данных и пустых результатов поиска.

Листинг 14 - фрагмент кода RequestHistoryWindow.xaml.cs

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows;
using System.Windows.Controls;

namespace ManagementCompanyApp
{
    public partial class RequestHistoryWindow : Window
    {
        private string connectionString;

        public RequestHistoryWindow(string connectionString)
        {
            InitializeComponent();
            this.connectionString = connectionString;
            LoadFilterData();
        }

        private void LoadFilterData()
        {
            try
            {
                // Загрузка сотрудников для фильтра
                string employeesQuery = @"SELECT EmployeeID,
                                           LastName + ' ' + FirstName AS
EmployeeName
                                           FROM Employees WHERE IsActive = 1
                                           ORDER BY LastName";

                using (SqlConnection connection = new
SqlConnection(connectionString))
                using (SqlCommand command = new SqlCommand(employeesQuery,
connection))
                using (SqlDataAdapter adapter = new
SqlDataAdapter(command))
                {
                    DataTable dataTable = new DataTable();
                    adapter.Fill(dataTable);

                    CbFilterValue.DisplayMemberPath = "EmployeeName";
                }
            }
        }
    }
}
```

```

        CbFilterValue.SelectedValuePath = "EmployeeID";
        CbFilterValue.ItemsSource = dataTable.DefaultView;

        if (dataTable.Rows.Count > 0)
            CbFilterValue.SelectedIndex = 0;
    }

    CbFilterType.SelectionChanged +=
CbFilterType_SelectionChanged;
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка загрузки данных фильтра:
{ex.Message}", "Ошибка");
    }
}

private void CbFilterType_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    if (CbFilterType.SelectedItem == null) return;

    ComboBoxItem selectedType =
(ComboBoxItem) CbFilterType.SelectedItem;
    string filterType = selectedType.Content.ToString();

    try
    {
        if (filterType == "Сотруднику")
        {
            // Загрузка сотрудников
            string employeesQuery = @"SELECT EmployeeID,
                                     LastName + ' ' + FirstName AS
DisplayName
                                     FROM Employees WHERE IsActive
                                     = 1
                                     ORDER BY LastName";

            FillFilterComboBox(employeesQuery, "DisplayName",
"EmployeeID");
        }
        else if (filterType == "Адресу")

```

```

        {
            // Загрузка адресов
            string addressesQuery = @"SELECT BuildingID, Address
AS DisplayName
                                FROM Buildings
                                ORDER BY Address";

            FillFilterComboBox(addressesQuery, "DisplayName",
"BuildingID");
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Ошибка переключения фильтра:
{ex.Message}", "Ошибка");
    }
}

private void FillFilterComboBox(string query, string
displayMember, string valueMember)
{
    try
    {
        using (SqlConnection connection = new
SqlConnection(connectionString))
        using (SqlCommand command = new SqlCommand(query,
connection))
        using (SqlDataAdapter adapter = new
SqlDataAdapter(command))
        {
            DataTable dataTable = new DataTable();
            adapter.Fill(dataTable);

            CbFilterValue.DisplayMemberPath = displayMember;
            CbFilterValue.SelectedValuePath = valueMember;
            CbFilterValue.ItemsSource = dataTable.DefaultView;

            if (dataTable.Rows.Count > 0)
                CbFilterValue.SelectedIndex = 0;
        }
    }
    catch (Exception ex)

```

```

        {
            MessageBox.Show($"Ошибка загрузки данных: {ex.Message}",
"Ошибка");
        }
    }

    private void BtnShow_Click(object sender, RoutedEventArgs e)
    {
        try
        {
            if (CbFilterType.SelectedItem == null ||
CbFilterValue.SelectedValue == null)
            {
                MessageBox.Show("Выберите тип фильтра и значение",
"Ошибка");

                return;
            }

            ComboBoxItem selectedType =
(ComboBoxItem) CbFilterType.SelectedItem;
            string filterType = selectedType.Content.ToString();
            int filterValue =
Convert.ToInt32(CbFilterValue.SelectedValue);

            string query = "";

            if (filterType == "Сотруднику")
            {
                query = $"
                SELECT
                    h.ActionDate,
                    r.RequestNumber,
                    b.Address,
                    h.ActionType,
                    h.NewStatus,
                    e.LastName + ' ' + e.FirstName AS
EmployeeName,
                    h.Description
                FROM RequestHistory h
                INNER JOIN RepairRequests r ON h.RequestID =
r.RequestID
            ";
            }
        }
        catch { }
    }
}

```

```

                INNER JOIN Buildings b ON r.BuildingID =
b.BuildingID

                INNER JOIN Employees e ON h.EmployeeID =
e.EmployeeID

                WHERE h.EmployeeID = @FilterValue
                ORDER BY h.ActionDate DESC";
            }
            else if (filterType == "Адрес")
            {
                query = $"
                SELECT
                    h.ActionDate,
                    r.RequestNumber,
                    b.Address,
                    h.ActionType,
                    h.NewStatus,
                    e.LastName + ' ' + e.FirstName AS
EmployeeName,
                    h.Description
                FROM RequestHistory h
                INNER JOIN RepairRequests r ON h.RequestID =
r.RequestID
                INNER JOIN Buildings b ON r.BuildingID =
b.BuildingID
                INNER JOIN Employees e ON h.EmployeeID =
e.EmployeeID

                WHERE r.BuildingID = @FilterValue
                ORDER BY h.ActionDate DESC";
            }

            using (SqlConnection connection = new
SqlConnection(connectionString))
            using (SqlCommand command = new SqlCommand(query,
connection))
            {
                command.Parameters.AddWithValue("@FilterValue",
filterValue);

                using (SqlDataAdapter adapter = new
SqlDataAdapter(command))
                {
                    DataTable dataTable = new DataTable();

```



```

<Grid Margin="10">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*/>
    </Grid.RowDefinitions>

    <Border Grid.Row="0" Background="#67BA80" Padding="10"
Margin="0,0,0,10" CornerRadius="5">
        <StackPanel>
            <TextBlock Text="Заявки на ремонт"
                FontSize="20"
                FontWeight="Bold"
                Foreground="White"
                HorizontalAlignment="Center"/>

            <WrapPanel Margin="0,10,0,0" HorizontalAlignment="Center">
                <ComboBox x:Name="CbStatus" Width="150" Margin="5">
                    <ComboBoxItem Content="Все статусы"
IsSelected="True"/>

                    <ComboBoxItem Content="Открыта"/>
                    <ComboBoxItem Content="В работе"/>
                    <ComboBoxItem Content="Заявка в работе"/>
                    <ComboBoxItem Content="Заявка закрыта"/>
                </ComboBox>

                <Button Content="Обновить"
                    Background="White"
                    Foreground="#67BA80"
                    Width="100"
                    Height="30"
                    Margin="5"
                    Click="BtnRefresh_Click"/>
            </WrapPanel>
        </StackPanel>
    </Border>

    <DataGrid x:Name="DataGridRequests"
        Grid.Row="1"
        Background="FFFFFF"
        AlternatingRowBackground="F4E8D3"
        AutoGenerateColumns="False"
        CanUserAddRows="False"

```

```

        CanUserDeleteRows="False"
        SelectionChanged="DataGridRequests_SelectionChanged">
        <DataGrid.Columns>
            <DataGridTextColumn Header="Номер" Binding="{Binding
RequestNumber}" Width="100"/>
            <DataGridTextColumn Header="Дата" Binding="{Binding
RequestDate, StringFormat=dd.MM.yyyy HH:mm}" Width="120"/>
            <DataGridTextColumn Header="Адрес" Binding="{Binding
Address}" Width="200"/>
            <DataGridTextColumn Header="Квартира" Binding="{Binding
ApartmentNumber}" Width="80"/>
            <DataGridTextColumn Header="Заявитель" Binding="{Binding
ApplicantFullName}" Width="180"/>
            <DataGridTextColumn Header="Проблема" Binding="{Binding
ProblemDescription}" Width="250"/>
            <DataGridTextColumn Header="Приоритет" Binding="{Binding
Priority}" Width="80"/>
            <DataGridTextColumn Header="Статус" Binding="{Binding
Status}" Width="120"/>
            <DataGridTextColumn Header="Исполнитель" Binding="{Binding
EmployeeName}" Width="150"/>
        </DataGrid.Columns>
        </DataGrid>
    </Grid>
</Window>

```

В листинге 16 представлен код C# окна заявок на ремонт. Этот код содержит логику для отображения и управления заявками на ремонт. Реализована фильтрация заявок по статусу, загрузка данных с сортировкой по приоритету и дате, а также функции редактирования и удаления заявок при выборе их в таблице. При удалении заявки выполняется проверка на наличие связанных записей в истории и запрашивается дополнительное подтверждение от пользователя.

Листинг 16 – RequestsWindow.xaml.cs

```

using System;
using System.Data;
using System.Data.SqlClient;

```

```

using System.Windows;
using System.Windows.Controls;

namespace ManagementCompanyApp
{
    public partial class RequestsWindow : Window
    {
        private string connectionString;

        public RequestsWindow(string connectionString)
        {
            InitializeComponent();
            this.connectionString = connectionString;
            LoadData();
        }

        private void LoadData()
        {
            try
            {
                string statusFilter = "";
                if (CbStatus.SelectedIndex > 0)
                {
                    ComboBoxItem item =
(ComboBoxItem)CbStatus.SelectedItem;
                    statusFilter = $" AND r.Status = '{item.Content}'";
                }

                string query = $"
                SELECT
                    r.RequestID,
                    r.RequestNumber,
                    r.RequestDate,
                    b.Address,
                    ISNULL(a.ApartmentNumber, '') AS ApartmentNumber,
                    r.ApplicantFullName,
                    r.ProblemDescription,
                    r.Priority,
                    r.Status,
                    ISNULL(e.LastName + ' ' + e.FirstName, 'Не
назначен') AS EmployeeName
                FROM RepairRequests r
            
```

```

INNER JOIN Buildings b ON r.BuildingID = b.BuildingID
LEFT JOIN Apartments a ON r.ApartmentID =
a.ApartmentID
LEFT JOIN Employees e ON r.AssignedToEmployeeID =
e.EmployeeID

WHERE 1=1 {statusFilter}
ORDER BY
CASE
    WHEN r.Priority = 1 THEN 1
    WHEN r.Priority = 2 THEN 2
    WHEN r.Priority = 3 THEN 3
    ELSE 4
END,
r.RequestDate DESC";

using (SqlConnection connection = new
SqlConnection(connectionString))
using (SqlCommand command = new SqlCommand(query,
connection))
using (SqlDataAdapter adapter = new
SqlDataAdapter(command))
{
    DataTable dataTable = new DataTable();
    adapter.Fill(dataTable);
    DataGridViewRequests.ItemsSource = dataTable.DefaultView;

    // Устанавливаем заголовок с количеством заявок
    int rowCount = dataTable.Rows.Count;
    Title = $"Заявки на ремонт ({rowCount} шт.)";
}
}
catch (SqlException sqlEx)
{
    MessageBox.Show($"Ошибка базы данных при загрузке данных:
{sqlEx.Message}\n\n" +
        "Проверьте подключение к базе данных.",
        "Ошибка базы данных",
        MessageBoxButton.OK,
        MessageBoxImage.Error);
}
catch (Exception ex)
{

```

```

        MessageBox.Show($"Ошибка загрузки данных: {ex.Message}",
                        "Ошибка загрузки",
                        MessageBoxButton.OK,
                        MessageBoxImage.Error);
    }
}

private void BtnRefresh_Click(object sender, RoutedEventArgs e)
{
    LoadData();
}

private void DataGridRequests_SelectionChanged(object sender,
SelectionChangedEventArgs e)
{
    try
    {
        if (DataGridRequests.SelectedItem != null)
        {
            DataRowView row =
(DataRowView)DataGridRequests.SelectedItem;
            int requestId = Convert.ToInt32(row["RequestID"]);
            string requestNumber =
row["RequestNumber"].ToString();

            // Спрашиваем пользователя, что он хочет сделать
            var result = MessageBox.Show($"Выберите действие для
заявки №{requestNumber}:\n\n" +
"Да - Редактировать
заявку\n" +
"Нет - Удалить заявку\n" +
"Отмена - Отменить выбор",
"Выбор действия",
MessageBoxButton.YesNoCancel,
MessageBoxImage.Question,
MessageBoxResult.Cancel);

            if (result == MessageBoxResult.Yes)
            {
                // Редактирование заявки
                EditRequest(requestId);
            }
        }
    }
}

```

```

        }
        else if (result == DialogResult.No)
        {
            // Удаление заявки
            DeleteRequest(requestId, requestNumber);
        }

        // Снимаем выделение
        DataGridViewRequests.SelectedItem = null;
    }
}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка при выборе заявки: {ex.Message}",
        "Ошибка выбора",
        MessageBoxButtons.OK,
        MessageBoxIcon.Error);
}
}

private void EditRequest(int requestId)
{
    try
    {
        var editWindow = new CreateRequestWindow(connectionString,
requestId);

        editWindow.Owner = this;
        bool? editResult = editWindow.ShowDialog();

        if (editResult == true)
        {
            LoadData(); // Обновляем список
            MessageBox.Show("Заявка успешно отредактирована!\n\n"
+
                "Изменения сохранены в системе.",
                "Успешное редактирование",
                MessageBoxButtons.OK,
                MessageBoxIcon.Information);
        }
    }
    catch (Exception ex)
    {

```

```

        MessageBox.Show($"Ошибка редактирования заявки:
{ex.Message}\n\n" +
        "Проверьте подключение к базе данных.",
        "Ошибка редактирования",
        MessageBoxButton.OK,
        MessageBoxImage.Error);
    }
}

private void DeleteRequest(int requestId, string requestNumber)
{
    try
    {
        // Дополнительное подтверждение для удаления
        var confirmResult = MessageBox.Show($"Вы уверены, что
хотите удалить заявку №{requestNumber}? \n\n" +
        "ВНИМАНИЕ: Это действие
невозможно отменить!\n" +
        "Все данные о заявке
будут удалены безвозвратно.",
        "Подтверждение
удаления",
        MessageBoxButton.YesNo,
        MessageBoxImage.Warning,
        MessageBoxResult.No);

        if (confirmResult != MessageBoxResult.Yes)
            return;

        using (SqlConnection connection = new
SqlConnection(connectionString))
        {
            connection.Open();

            // Сначала проверяем, существует ли заявка
            string checkQuery = "SELECT COUNT(*) FROM
RepairRequests WHERE RequestID = @RequestID";
            using (SqlCommand checkCommand = new
SqlCommand(checkQuery, connection))
            {
                checkCommand.Parameters.AddWithValue("@RequestID",

```



```

requestId);

int exists =
Convert.ToInt32(checkCommand.ExecuteScalar());

if (exists == 0)
{
    MessageBox.Show("Заявка не найдена в базе
данных.\n\n" +
        "Возможно, она уже была
удалена другим пользователем.",
        "Заявка не найдена",
        MessageBoxButton.OK,
        MessageBoxImage.Information);

    LoadData();
    return;
}

// Удаляем заявку
string deleteQuery = "DELETE FROM RepairRequests WHERE
RequestID = @RequestID";
using (SqlCommand command = new
SqlCommand(deleteQuery, connection))
{
    command.Parameters.AddWithValue("@RequestID",
requestId);

    int rowsAffected = command.ExecuteNonQuery();

    if (rowsAffected > 0)
    {
        MessageBox.Show($"Заявка №{requestNumber}
успешно удалена!",
            "Успешное удаление",
            MessageBoxButton.OK,
            MessageBoxImage.Information);

        LoadData(); // Обновляем список
    }
    else
    {
        MessageBox.Show("Не удалось удалить
заявку.\n\n" +
            "Попробуйте обновить список и

```

```

повторить операцию.",

                                "Ошибка удаления",
                                MessageBoxButton.OK,
                                MessageBoxImage.Error);

                                }

                                }

                                }

                                }
catch (SqlException sqlEx) when (sqlEx.Number == 547)
{
    MessageBox.Show("Невозможно удалить заявку.\n\n" +
                    "Заявка имеет связанные записи в
истории.\n" +
                    "Сначала удалите связанные записи истории
заявок.",

                    "Ошибка удаления",
                    MessageBoxButton.OK,
                    MessageBoxImage.Error);

}
catch (SqlException sqlEx)
{
    MessageBox.Show($"Ошибка базы данных при удалении:
{sqlEx.Message}\n\n" +
                    $"Код ошибки: {sqlEx.Number}",
                    "Ошибка базы данных",
                    MessageBoxButton.OK,
                    MessageBoxImage.Error);

}
catch (Exception ex)
{
    MessageBox.Show($"Ошибка удаления заявки: {ex.Message}",
                    "Ошибка удаления",
                    MessageBoxButton.OK,
                    MessageBoxImage.Error);

}

}

}

}

```

В листинге 17 представлена разметка XAML окна статистики. Этот код описывает интерфейс для отображения статистики выполнения заявок. Окно содержит три информационных блока с ключевыми показателями: общее

количество заявок, количество выполненных и находящихся в работе, а также таблицу со статистикой распределения заявок по приоритетам. Кнопка "Обновить статистику" позволяет актуализировать данные.

Листинг 17 – фрагмент кода StatisticsWindow.xaml

```
<Window x:Class="ManagementCompanyApp.StatisticsWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Статистика"
        Height="500"
        Width="800"
        WindowStartupLocation="CenterOwner"
        FontFamily="Segoe UI"
        Background="#FFFFFF"

Icon="C:\Users\User\source\repos\ManagementCompanyApp\Res\icon.ico">

    <Grid Margin="10">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
        </Grid.RowDefinitions>

        <Border Grid.Row="0" Background="#67BA80" Padding="10"
Margin="0,0,0,10" CornerRadius="5">
            <TextBlock Text="Статистика выполнения заявок"
                FontSize="20"
                FontWeight="Bold"
                Foreground="White"
                HorizontalAlignment="Center"/>
        </Border>

        <Grid Grid.Row="1">
            <Grid.RowDefinitions>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="Auto"/>
                <RowDefinition Height="*/>
            </Grid.RowDefinitions>

            <WrapPanel Grid.Row="0" HorizontalAlignment="Center">
                <Border Background="#F4E8D3" Margin="5" Padding="20"
CornerRadius="5">
```

```

        <StackPanel>
            <TextBlock Text="Всего заявок:"
                FontSize="14"
                FontWeight="Bold"/>
            <TextBlock x:Name="TxtTotalCount"
                Text="0"
                FontSize="36"
                FontWeight="Bold"
                Foreground="#67BA80"
                HorizontalAlignment="Center"/>
        </StackPanel>
    </Border>

    <Border Background="#F4E8D3" Margin="5" Padding="20"
        CornerRadius="5">
        <StackPanel>
            <TextBlock Text="Выполнено:"
                FontSize="14"
                FontWeight="Bold"/>
            <TextBlock x:Name="TxtCompletedCount"
                Text="0"
                FontSize="36"
                FontWeight="Bold"
                Foreground="#67BA80"
                HorizontalAlignment="Center"/>
        </StackPanel>
    </Border>

    <Border Background="#F4E8D3" Margin="5" Padding="20"
        CornerRadius="5">
        <StackPanel>
            <TextBlock Text="В работе:"
                FontSize="14"
                FontWeight="Bold"/>
            <TextBlock x:Name="TxtInProgressCount"
                Text="0"
                FontSize="36"
                FontWeight="Bold"
                Foreground="#67BA80"
                HorizontalAlignment="Center"/>
        </StackPanel>
    </Border>

```

```

        </WrapPanel>

        <Button Grid.Row="1"
            Content="Обновить статистику"
            Background="#67BA80"
            Foreground="White"
            Width="200"
            Height="35"
            Margin="0,10,0,10"
            HorizontalAlignment="Center"
            Click="BtnRefresh_Click"/>

        <Border Grid.Row="2"
            Background="#F4E8D3"
            Padding="10"
            CornerRadius="5">
            <StackPanel>
                <TextBlock Text="Статистика по приоритетам:"
                    FontSize="16"
                    FontWeight="Bold"
                    Margin="0,0,0,10"/>

                <DataGrid x:Name="DataGridPriorityStats"
                    Height="200"
                    Background="White"
                    AutoGenerateColumns="False">
                    <DataGrid.Columns>
                        <DataGridTextColumn Header="Приоритет"
Binding="{Binding Priority}" Width="100"/>
                        <DataGridTextColumn Header="Количество заявок"
Binding="{Binding Count}" Width="150"/>
                        <DataGridTextColumn Header="Процент"
Binding="{Binding PercentageString}" Width="100"/>
                    </DataGrid.Columns>
                </DataGrid>
            </StackPanel>
        </Border>
    </Grid>
</Grid>
</Window>

```

В листинге 18 представлен код C# окна статистики. Этот код содержит логику для сбора и отображения статистических данных по заявкам на ремонт.

Выполняются SQL-запросы для получения общего количества заявок, количества выполненных и находящихся в работе заявок, а также статистики распределения заявок по приоритетам с расчетом процентного соотношения. Данные выводятся в текстовые блоки и таблицу, обновляются по нажатию кнопки "Обновить статистику".

Листинг 18 – фрагмент кода StatisticsWindow.xaml.cs

```
using System;
using System.Data;
using System.Data.SqlClient;
using System.Windows;

namespace ManagementCompanyApp
{
    public partial class StatisticsWindow : Window
    {
        private string connectionString;

        public StatisticsWindow(string connectionString)
        {
            InitializeComponent();
            this.connectionString = connectionString;
            LoadStatistics();
        }

        private void LoadStatistics()
        {
            try
            {
                using (SqlConnection connection = new
SqlConnection(connectionString))
                {
                    connection.Open();
                    string query1 = "SELECT COUNT(*) FROM RepairRequests";
                    using (SqlCommand command = new SqlCommand(query1,
connection))
                    {
                        int totalCount =
Convert.ToInt32(command.ExecuteScalar());
                        TxtTotalCount.Text = totalCount.ToString();
                    }
                }
            }
        }
    }
}
```

```

        // Количество выполненных заявок
        string query2 = "SELECT COUNT(*) FROM RepairRequests
WHERE Status = 'Заявка закрыта'";
        using (SqlCommand command = new SqlCommand(query2,
connection))
        {
            int completedCount =
Convert.ToInt32(command.ExecuteScalar());
            TxtCompletedCount.Text =
completedCount.ToString();
        }

        // Количество заявок в работе
        string query3 = "SELECT COUNT(*) FROM RepairRequests
WHERE Status IN ('В работе', 'Заявка в работе')";
        using (SqlCommand command = new SqlCommand(query3,
connection))
        {
            int inProgressCount =
Convert.ToInt32(command.ExecuteScalar());
            TxtInProgressCount.Text =
inProgressCount.ToString();
        }

        // Статистика по приоритетам
        string query4 = @"
        SELECT
            Priority,
            COUNT(*) as Count,
            CAST(COUNT(*) * 100.0 / (SELECT COUNT(*) FROM
RepairRequests) AS DECIMAL(5,2)) as Percentage
        FROM RepairRequests
        GROUP BY Priority
        ORDER BY Priority";

        using (SqlCommand command = new SqlCommand(query4,
connection))

        using (SqlDataAdapter adapter = new
SqlDataAdapter(command))
        {
            DataTable dataTable = new DataTable();

```

```

        adapter.Fill(dataTable);

        // Добавляем столбец со строковым представлением
процента

        dataTable.Columns.Add("PercentageString",
typeof(string));

        foreach (DataRow row in dataTable.Rows)
        {
            if (row["Percentage"] != DBNull.Value)
            {
                row["PercentageString"] =
string.Format("{0:N2}%", row["Percentage"]);
            }
            else
            {
                row["PercentageString"] = "0.00%";
            }
        }

        DataGridPriorityStats.ItemsSource =
dataTable.DefaultView;
    }

}

catch (Exception ex)
{
    MessageBox.Show($"Ошибка загрузки статистики:
{ex.Message}", "Ошибка");
}

private void BtnRefresh_Click(object sender, RoutedEventArgs e)
{
    LoadStatistics();
}
}
}

```

На рисунке 6 показано главное меню приложения для управляющей компании. В нём есть кнопки для работы с заявками, жилым фондом, квартирами, сотрудниками, историей и статистикой. Внизу — кнопки «Новая заявка» (чтобы создать новую) и «Назад». Справа внизу отображается статус подключения к системе и кнопка для его проверки.

Рисунок 6 – Главное окно

На рисунке 7 показан экран со списком заявок на ремонт. Вверху есть фильтр по статусу и кнопка «Обновить». В таблице можно увидеть номер, дату, адрес, квартиру, кто оставил заявку, описание проблемы, приоритет, текущий статус и кто назначен исполнителем.

Заявки на ремонт (10 шт.)

Заявки на ремонт

Все статусы Обновить

Номер	Дата	Адрес	Квартира	Заявитель	Проблема	Приоритет	Статус	Исполнитель
Z-20260117-1846	16.01.2026 14:41	ул. Матросова, 179а, Светлоград	1	Шеченко Ольга Викторовна	Протекает кран на кухне	1	Заявка закрыта	Иванова Мария
Z-20260117-2134	13.01.2026 14:41	ул. Матросова, 179а, Светлоград	4	Савельев Олег Иванович	Трещина в стене на балконе	1	В работе	Иванова Мария
Z-20260117-8403	15.01.2026 14:41	ул. Матросова, 179а, Светлоград	2	Мазалова Ирина Львовна	Не работает розетка в ванной	2	Заявка закрыта	Сидоров Дмитрий
Z-20260117-2340	12.01.2026 14:41	ул. Матросова, 179а, Светлоград	5	Габиец Игорь Леонидович	Не закрывается входная дверь	2	В работе	Сидоров Дмитрий
Z-20260117-4951	14.01.2026 14:41	ул. Матросова, 179а, Светлоград	3	Семенова Юрий Геннадиевич	Забит слив в раковине	3	Заявка закрыта	Петров Алексей
Z-20260117-5822	11.01.2026 14:41	ул. Матросова, 179а, Светлоград	6	Бунин Эдуард Михайлович	Протекает радиатор отопления	3	В работе	Петров Алексей
Z-20260117-7262	10.01.2026 14:41	ул. Матросова, 179а, Светлоград	7	Бахшиев Павел Инокентьевич	Сломан выключатель в коридоре	3	Открыта	Иванова Мария
Z-20260117-8386	09.01.2026 14:41	ул. Матросова, 179а, Светлоград	8	Байчорова Агата Рустамовна	Запах из канализационного стояка	3	Открыта	Сидоров Дмитрий
Z-20260117-2216	08.01.2026 14:41	ул. Матросова, 179а, Светлоград	9	Тюреникова Наталья Сергеевна	Отклеиваются обои в комнате	3	Открыта	Петров Алексей
Z-20260117-3323	07.01.2026 14:41	ул. Матросова, 179а, Светлоград	10	Александров Петр Константинос	Общее обследование помещения	3	Открыта	Иванова Мария

Рисунок 7 – Экран со списком заявок на ремонт

На рисунке 8 показан экран с отфильтрованным списком заявок на ремонт. В таблице отображаются заявки, которые находятся в процессе выполнения.

Заявки на ремонт (3 шт.)

Заявки на ремонт

В работе Обновить

Номер	Дата	Адрес	Квартира	Заявитель	Проблема	Приоритет	Статус	Исполнитель
Z-20260117-2134	13.01.2026 14:41	ул. Матросова, 179а, Светлоград	4	Савельев Олег Иванович	Трещина в стене на балконе	1	В работе	Иванова Мария
Z-20260117-2340	12.01.2026 14:41	ул. Матросова, 179а, Светлоград	5	Габиец Игорь Леонидович	Не закрывается входная дверь	2	В работе	Сидоров Дмитрий
Z-20260117-5822	11.01.2026 14:41	ул. Матросова, 179а, Светлоград	6	Бунин Эдуард Михайлович	Протекает радиатор отопления	3	В работе	Петров Алексей

Рисунок 8 – Фильтрация списка

На рисунке 9 показан экран с данными жилого фонда. В нём отображается таблица с информацией о всех домах. В таблице есть данные по каждому дому: адрес, количество этажей и квартир, год постройки, общая площадь, дата начала управления и его текущий статус. Все дома в списке имеют статус "Активен".

Адрес	Этажей	Квартир	Год постройки	Площадь, м²	Управление с	Статус
пл. Выставочная, 40, Светлоград	5	70	1985	2,369.2	01.12.2018	Активен
пл. Выставочная, 43, Светлоград	5	68	1987	3,702.8	01.07.2019	Активен
пл. Выставочная, 45, Светлоград	4	68	1990	3,731.5	01.12.2019	Активен
пл. Выставочная, 47, Светлоград	5	68	1993	4,079.2	01.07.2019	Активен
пл. Выставочная, 48, Светлоград	5	68	1995	3,654.6	01.12.2018	Активен
пл. Выставочная, 49, Светлоград	5	60	1995	2,891.7	01.06.2021	Активен
пл. Выставочная, 50, Светлоград	5	60	1995	4,014.5	01.02.2019	Активен
пл. Выставочная, 57, Светлоград	3	36	2015	2,075.7	01.02.2020	Активен
пл. Выставочная, 58, Светлоград	5	0	2013	3,124.2	01.11.2021	Активен
ул. 45 Параллель, 4/2, Ставрополь	9	52	2001	1,978.7	22.04.2015	Активен
ул. Бассейная, 82, Светлоград	5	48	1988	3,255.3	01.03.2021	Активен
ул. Васильева, 1, Ставрополь	9	144	1983	7,950.4	22.04.2015	Активен
ул. Доваторцев, 66/2, Ставрополь	9	102	1984	3,176.4	01.11.2020	Активен
ул. Красная, 44а, Светлоград	5	60	1983	4,317.4	01.03.2022	Активен
ул. Матросова, 179а, Светлоград	4	28	1979	879.4	18.02.2022	Активен
ул. Мира, 23б, Ставрополь	10	62	1991	4,423.1	04.11.2007	Активен
ул. Мира, 272, Ставрополь	9	88	2006	6,204.7	22.04.2015	Активен
ул. Мира, 278, Ставрополь	10	40	2008	3,294.1	01.08.2019	Активен
ул. Пушкина, 12, Светлоград	5	118	1980	5,316.8	01.10.2020	Активен
ул. Пушкина, 3а, Светлоград	5	48	1990	1,007.9	01.12.2018	Активен
ул. Яматорская, 21, Светлоград	5	58	1985	2,586.6	01.01.2021	Активен

Рисунок 9 – Экран с данными жилого фонда

На рисунке 10 показан экран выбора действия для заявки на ремонт. Вверху отображается подробная информация о конкретной заявке: адрес, номер квартиры, заявитель и описание проблемы. В центре экрана находится меню выбора действия с предложением "Редактировать заявку", "Удалить заявку" или "Отменить выбор". В нижней части экрана есть текстовое поле для ввода описания проблемы с подсказками-вариантами.

в	Адрес	Квартира	Заявитель	Проблема
1.2026 14:41	ул. Матросова, 179а, Светлоград	1	Шевченко Ольга Викторовна	Протекает кран на
1.2026 14:41	ул. Матросова, 179а,			ещина в стене на
1.2026 14:41	ул. Матросова, 179а,			работает розетку
1.2026 14:41	ул. Матросова, 179а,			закрывается вхо
1.2026 14:41	ул. Матросова, 179а,			бит слив в раков
1.2026 14:41	ул. Матросова, 179а,			отекает радиатор
1.2026 14:41	ул. Матросова, 179а,			оман выключате
1.2026 14:41	ул. Матросова, 179а,			пах из канализац
1.2026 14:41	ул. Матросова, 179а,			клеиваются обои
1.2026 14:41	ул. Матросова, 179а,			ещее обследован

Выбор действия

Выберите действие для заявки №2-20260117-1846:

Да - Редактировать заявку
 Нет - Удалить заявку
 Отмена - Отменить выбор

Да Нет Отмена

Рисунок 10 - Экран выбора действия для заявки на ремонт

На рисунке 11 показан экран создания новой заявки.

Создание новой заявки на ремонт

Адрес дома:*
ул. Матросова, 179а, Светлоград

Номер квартиры (опционально):
1

ФИО заявителя:*
Шевченко Ольга Викторовна

Контактный телефон:*
+79180000001

Описание проблемы:*
Протекает кран на кухне

Ответственный исполнитель:

Отмена Сохранить

Рисунок 11 - Экран создания новой заявки

На рисунке 12 показан экран с информацией о квартирах. Вверху можно выбрать конкретный дом из списка и нажать кнопку «Показать». Ниже отображается таблица с данными по квартирам выбранного дома. Все квартиры в примере имеют статус "Заселена".

Квартиры

Выберите дом: ул. Матросова, 179а, Светлоград Показать

№ квартиры	Владелец	Телефон	Площадь, м²	Комнат	Статус
1	Шевченко Ольга Викторовна	+79180000001	45.5	2	Заселена
2	Мазалова Ирина Львовна	+79180000002	48.2	2	Заселена
3	Семениха Юрий Геннадьевич	+79180000003	42.8	1	Заселена
4	Савельев Олег Иванович	+78207815445	51.3	3	Заселена
5	Габичев Игорь Леонидович	+79180000005	47.6	2	Заселена
6	Бунин Эдуард Михайлович	+79180000006	43.9	2	Заселена
7	Бахшиев Павел Иннокентьевич	+79180000007	39.8	1	Заселена
8	Байнорова Агата Рустамовна	+89643324574	52.1	3	Заселена
9	Тюреникова Наталья Сергеевна	+89629987214	46.7	2	Заселена
10	Александров Петр Константинович	+79180000010	44.3	2	Заселена
11	Мазалова Ольга Николаевна	+79180000011	50.2	3	Заселена
12	Лапшин Виктор Романович	+79180000012	41.5	1	Заселена
13	Гусев Семен Петрович	+89188601163	49.8	3	Заселена
14	Гладилина Вера Михайловна	+79180000014	47.2	2	Заселена
15	Лукин Илья Федорович	+89634568714	43.6	2	Заселена
16	Петров Станислав Игоревич	+8918818187845	52.4	3	Заселена
17	Филь Марина Федоровна	+79180000017	40.9	1	Заселена
18	Михайлов Игорь Владимирович	+79180000018	46.1	2	Заселена
19	Масок Динара Викторовна	+79180000019	48.9	3	Заселена
20	Мартыненко Александр Сергеевич	+89183215428	45.7	2	Заселена
21	Устьянцева Анна Станиславовна	+79180000021	42.3	2	Заселена
22	Антоненко Дмитрий Игоревич	+79180000022	51.8	3	Заселена
23	Любашева Галина Аркадьевна	+89625674581	44.5	2	Заселена
24	Захарьев Денис Сергеевич	+79180000024	47.9	2	Заселена
25	Третьяк Ярослав Викторовна	+79180000025	43.2	2	Заселена
26	Бондарь Сергей Владимирович	+79180000026	49.5	3	Заселена
27	Петраков Артем Сергеевич	+79180000027	41.8	1	Заселена
28	Вальке Рита Владимировна	+79180000028	46.3	2	Заселена

Рисунок 12 – Экран с информацией о квартирах

На рисунке 13 показан экран со списком сотрудников. В таблице представлена информация о каждом сотруднике: фамилия, имя, отчество, должность, отдел, контактный телефон и электронная почта. Внизу экрана есть кнопки для переключения между страницами списка «<» и «>».

Фамилия	Имя	Отчество	Должность	Отдел	Телефон	Email
Васильев	Сергей	Петрович	Управляющий	Администрация	+79161234505	vasiliev@company.ru
Иванова	Мария	Сергеевна	Электрик	Ремонтная служба	+79161234502	ivanova@company.ru
Козлова	Ольга	Викторовна	Диспетчер	Приемная	+79161234504	kozlova@company.ru
Николаев	Андрей	Михайлович	Мастер	Ремонтная служба	+79161234506	nikolaev@company.ru
Петров	Алексей	Иванович	Слесарь	Ремонтная служба	+79161234501	petrov@company.ru
Сидоров	Дмитрий	Александрович	Сантехник	Ремонтная служба	+79161234503	sidorov@company.ru
Федорова	Елена	Александровна	Бухгалтер	Финансовый отдел	+79161234507	fedorova@company.ru

Рисунок 13 – Экран со списком сотрудников

На рисунке 14 показан экран истории заявок. Вверху можно отфильтровать записи по адресу и нажать кнопку «Показать». Ниже отображается таблица с историей, где для каждой заявки указаны: дата, номер заявки, адрес, тип действия, статус, сотрудник и описание действия. Внизу экрана находятся кнопки «Назад» для возврата в меню.

Дата	Номер заявки	Адрес	Тип действия	Статус	Сотрудник	Описание
16.01.2026 14:41	Z-20260117-1846	ул. Матросова, 179а, Светлоград	Создание заявки	Заявка закрыта	Иванова Мария	Заявка создана диспетчером
15.01.2026 14:41	Z-20260117-8403	ул. Матросова, 179а, Светлоград	Создание заявки	Заявка закрыта	Сидоров Дмитрий	Заявка создана диспетчером
14.01.2026 14:41	Z-20260117-4951	ул. Матросова, 179а, Светлоград	Создание заявки	Заявка закрыта	Петров Алексей	Заявка создана диспетчером
13.01.2026 14:41	Z-20260117-2134	ул. Матросова, 179а, Светлоград	Создание заявки	В работе	Иванова Мария	Заявка создана диспетчером
12.01.2026 14:41	Z-20260117-2340	ул. Матросова, 179а, Светлоград	Создание заявки	В работе	Сидоров Дмитрий	Заявка создана диспетчером
11.01.2026 14:41	Z-20260117-5822	ул. Матросова, 179а, Светлоград	Создание заявки	В работе	Петров Алексей	Заявка создана диспетчером
10.01.2026 14:41	Z-20260117-7262	ул. Матросова, 179а, Светлоград	Создание заявки	Открыта	Иванова Мария	Заявка создана диспетчером
09.01.2026 14:41	Z-20260117-8388	ул. Матросова, 179а, Светлоград	Создание заявки	Открыта	Сидоров Дмитрий	Заявка создана диспетчером
08.01.2026 14:41	Z-20260117-2218	ул. Матросова, 179а, Светлоград	Создание заявки	Открыта	Петров Алексей	Заявка создана диспетчером
07.01.2026 14:41	Z-20260117-3323	ул. Матросова, 179а, Светлоград	Создание заявки	Открыта	Иванова Мария	Заявка создана диспетчером

Рисунок 14 - Экран истории заявок

На рисунке 15 показан экран статистики по заявкам. Вверху отображаются общие цифры: всего заявок, выполнено и в работе. Ниже расположена кнопка «Обновить статистику». В центральной части экрана приведена детальная статистика по приоритетам заявок в виде таблицы. Внизу находится кнопка «Назад» для возврата в меню.

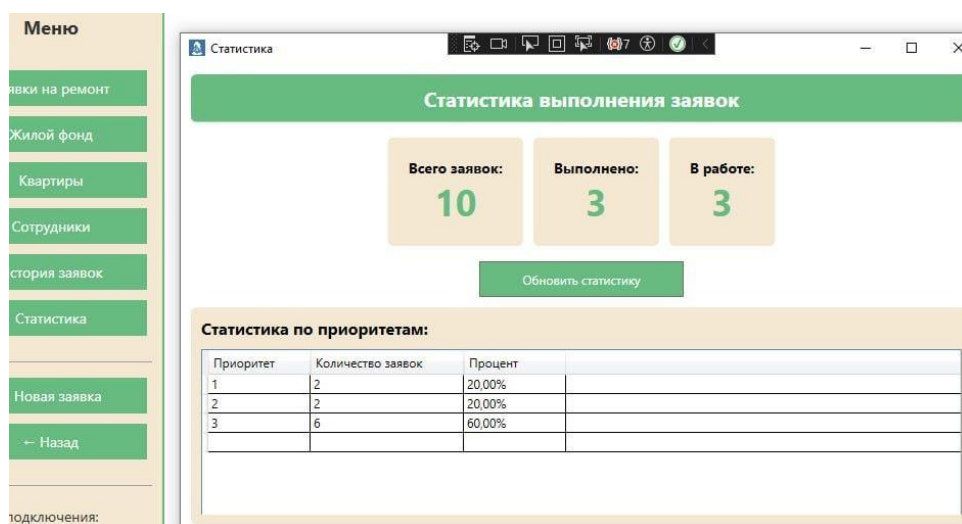


Рисунок 15 – Экран статистики по заявкам

На рисунке 16 показан экран успешного подключения к базе данных. Внизу появляется информационное сообщение, которое подтверждает, что подключение установлено. В нём указаны название сервера и базы данных.

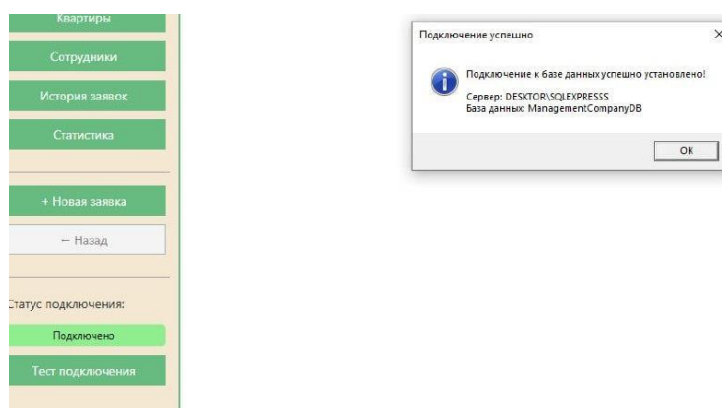


Рисунок 16 - экран успешного подключения к базе данных

На рисунке 17 показан экран выбора приоритета заявки. Отображается шкала или выпадающий список с уровнями приоритета от 1 (Высокий) до 5 (Низкий). В данном случае выбран средний приоритет — 2.

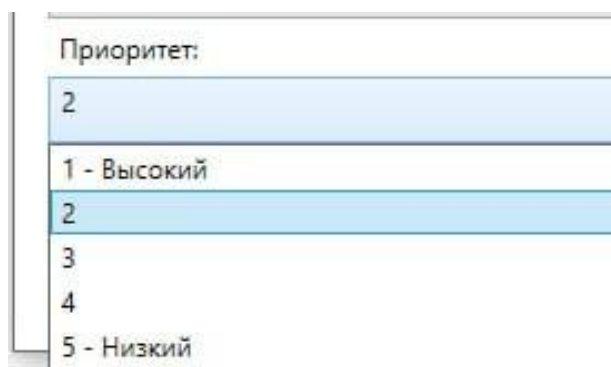


Рисунок 17 - экран выбора приоритета заявки

На рисунке 18 показан экран создания заявки с полями для заполнения. Поля включают адрес дома, номер квартиры, ФИО заявителя, контактный телефон, описание проблемы. Также здесь есть выбор ответственного исполнителя из выпадающего списка сотрудников.

Создание новой заявки на ремонт

Адрес дома*
пл. Выставочная, 40, Светлоград

Номер квартиры (опционально):
5

ФИО заявителя*
Хайретдинова Алина Ильясовна

Контактный телефон*
79265743348

Описание проблемы*
Протекает кран

Ответственный исполнитель:
Васильев Сергей Петрович

Васильев Сергей Петрович
Иванова Мария Сергеевна
Коллова Ольга Викторовна
Никольев Андрей Михайлович
Петров Алексей Иванович
Сидоров Дмитрий Александрович
Федорова Елена Александровна

Отмена Создать заявку

Рисунок 18 - экран создания заявки с полями для заполнения

На рисунке 19 показан экран выбора адреса дома при создании заявки. Вверху отображается выбранный адрес. Ниже представлен список всех доступных адресов для выбора. Внизу экрана, вероятно, находится поле или кнопка для указания или подтверждения статуса заявки.

Создание новой заявки на ремонт

Адрес дома*
пл. Выставочная, 40, Светлоград

пл. Выставочная, 40, Светлоград
пл. Выставочная, 43, Светлоград
пл. Выставочная, 45, Светлоград
пл. Выставочная, 47, Светлоград
пл. Выставочная, 48, Светлоград
пл. Выставочная, 49, Светлоград
пл. Выставочная, 50, Светлоград
пл. Выставочная, 57, Светлоград
пл. Выставочная, 58, Светлоград
ул. 45 Параллель, 4/2, Ставрополь
ул. Бассейная, 82, Светлоград
ул. Васильева, 1, Ставрополь
ул. Доваторцев, 66/2, Ставрополь
ул. Красная, 44а, Светлоград
ул. Матросова, 179а, Светлоград
ул. Мира, 236, Ставрополь
ул. Мира, 272, Ставрополь
ул. Мира, 278, Ставрополь

Статус:

Рисунок 19 - Экран выбора адреса дома при создании заявки

На рисунке 20 показана валидация поля "Контактный телефон". Под полем ввода появляется сообщение об ошибке или подсказка: «Телефон должен содержать только цифры, начинаться с +7 или 8». Ниже находится поле для описания проблемы.

Контактный телефон:*	
79265743348	
Описание проблем	Телефон должен содержать только цифры, начинаться с +7 или 8
Протекает кран	

Рисунок 20 - валидация поля "Контактный телефон"